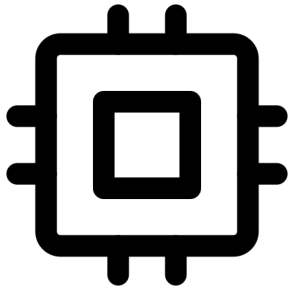
Getting Started with MANGO BLOCK

Control your board with Python

Codpy Section



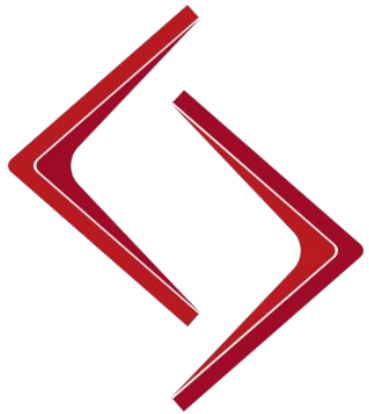
Morning: Su Li-Yuan

Afternoon: Cheng Yuan-Kai

SPONSOR



國立成功大學資訊工程學系暨研究所



敏求智慧運算學院

Miin Wu School of Computing



社團法人中華民國愛自造者學習協會



NOVATEK

SPONSOR

國立成功大學資訊工程學系暨研究所

敏求智慧運算學院
Miin Wu School of Computing

PROGRAM the WORLD
社團法人中華民國愛自造者學習協會

NOVATEK

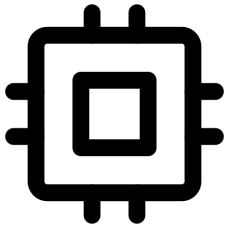
Today's Schedule

09:00	Welcome & the board	
09:20	Setup	Lab
10:00	LEDs & buttons	Lab
10:30	Break	
10:40	Buzzer	Lab
11:00	Web dashboard project	Lab
11:45	Demos & wrap-up	
13:00	Servo control & comms	Lab
14:30	Multiple servos	Lab



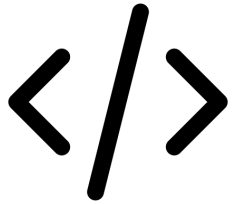
Course handbook

Today's Goals



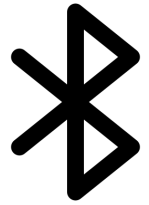
Know the board

LEDs, buzzer, buttons:
where they are



Control it with code

Output: light, sound
Input: buttons



Control it from the web

Bluetooth web dashboard
done by lunch

The Finished Project



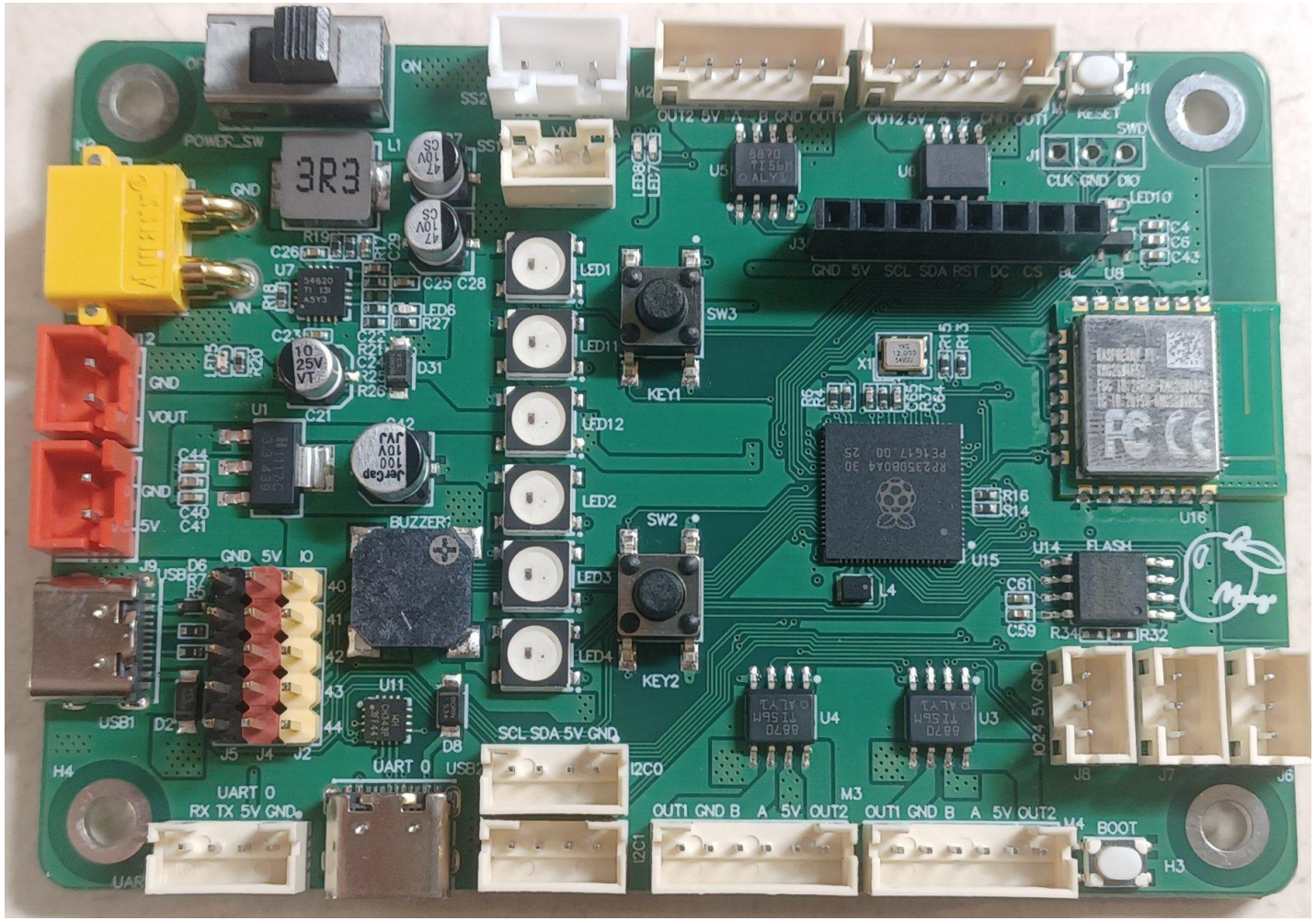
Part 0

Meet MANGO BLOCK

What's on the board

09:05 – 09:20

This Is MANGO BLOCK



MANGO BLOCK

Motor driver board

One board = a tiny computer

Runs Python (MicroPython)

Has Bluetooth and Wi-Fi

Drives servos (afternoon)

The Two Main Chips

RP2350B

The brain

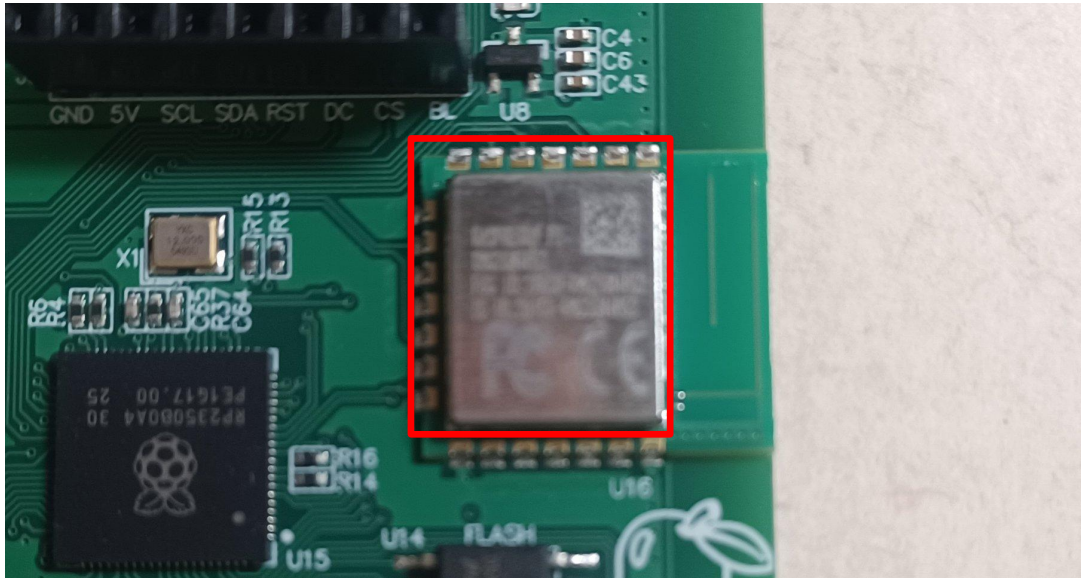
Runs the code you write
Controls LEDs, buzzer, buttons, servos



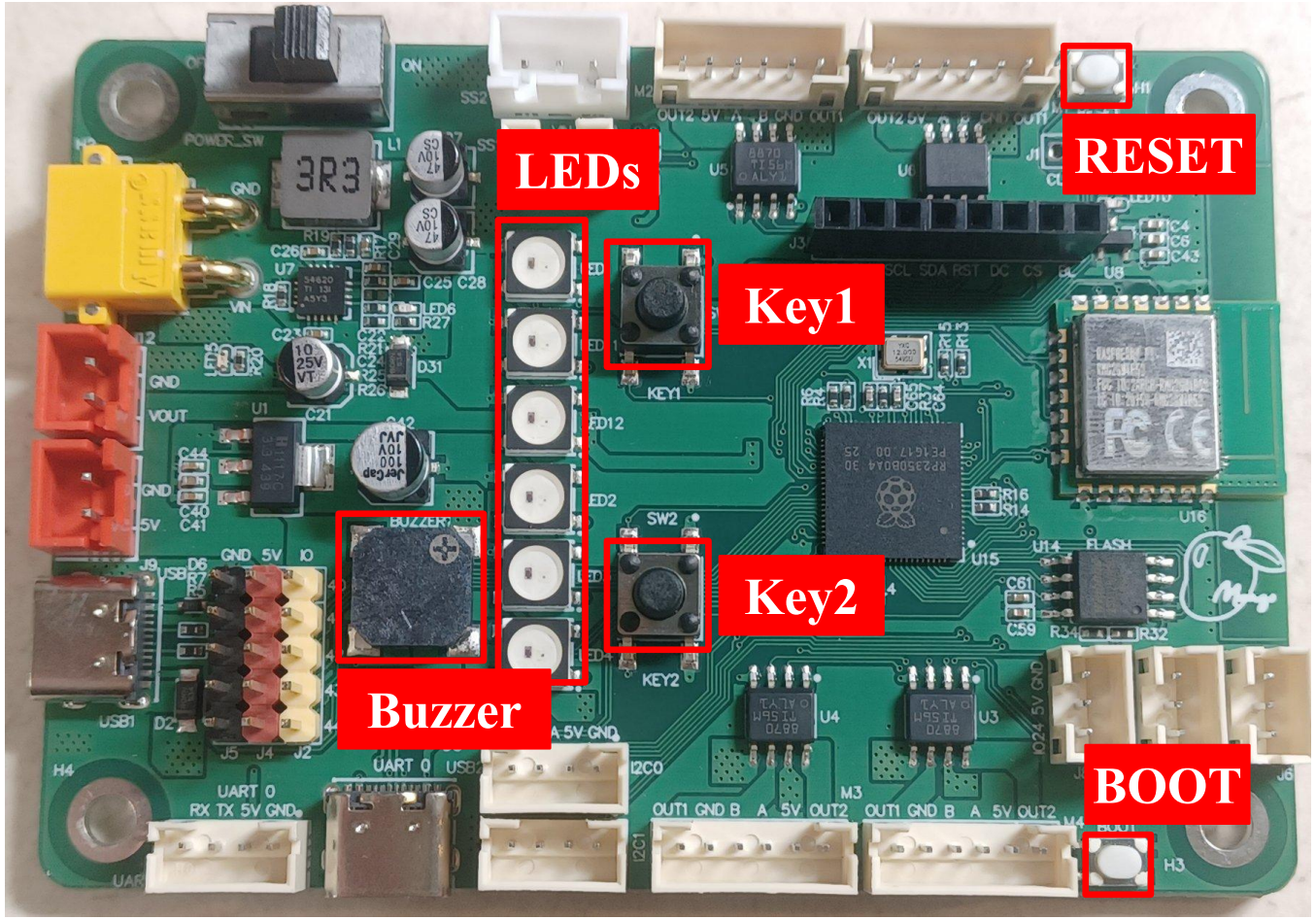
RM2

Mouth and ears

Bluetooth and Wi-Fi
Today's project uses it to talk to the web page



Parts We Use Today



6 RGB LEDs

WS2812B, any color you like

Buzzer

Plays different pitches

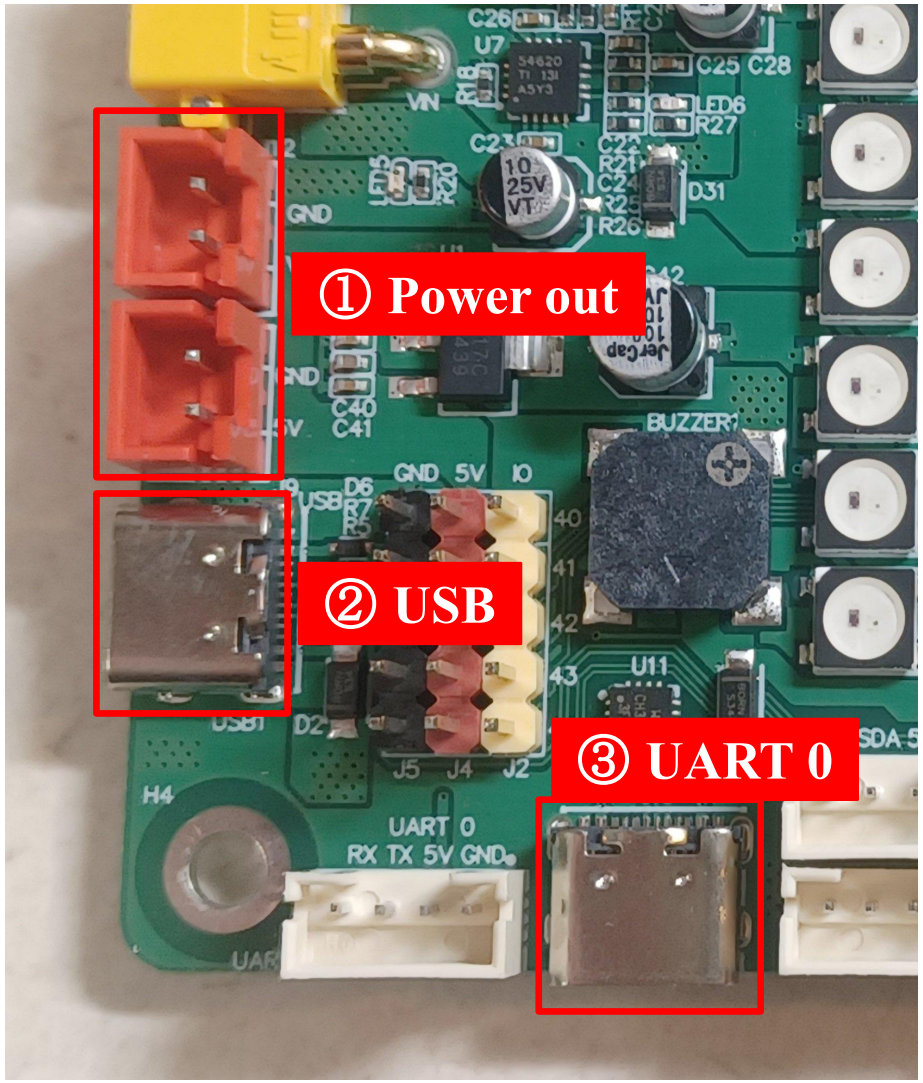
Key1 / Key2

Two buttons for you

BOOT / RESET

Flash firmware, restart

Use This Type-C Port Only



① Red connectors: power out

GND / VOUT / 5V, powers other modules
Not for your computer

② USB (Type-C): to computer

Power + upload code
Use this port all day

③ UART 0 (Type-C): not the same

Different protocol; plugged into a PC
it can't upload code

To your computer: always use ② USB

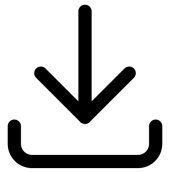
Part 1

Setup

Download · Flash firmware · Python · VS Code · MicroPico

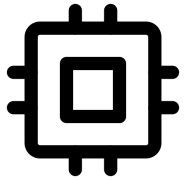
09:20 – 10:00

What You Need



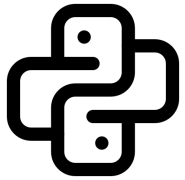
Course files

GitHub →
Download ZIP



Course firmware

Course
.uf2 file



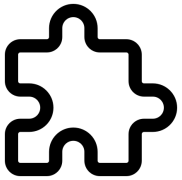
Python

Runs the
dashboard web page



VS Code

Where you write
code



MicroPico

VS Code extension
Talks to the board

Download the Course Files

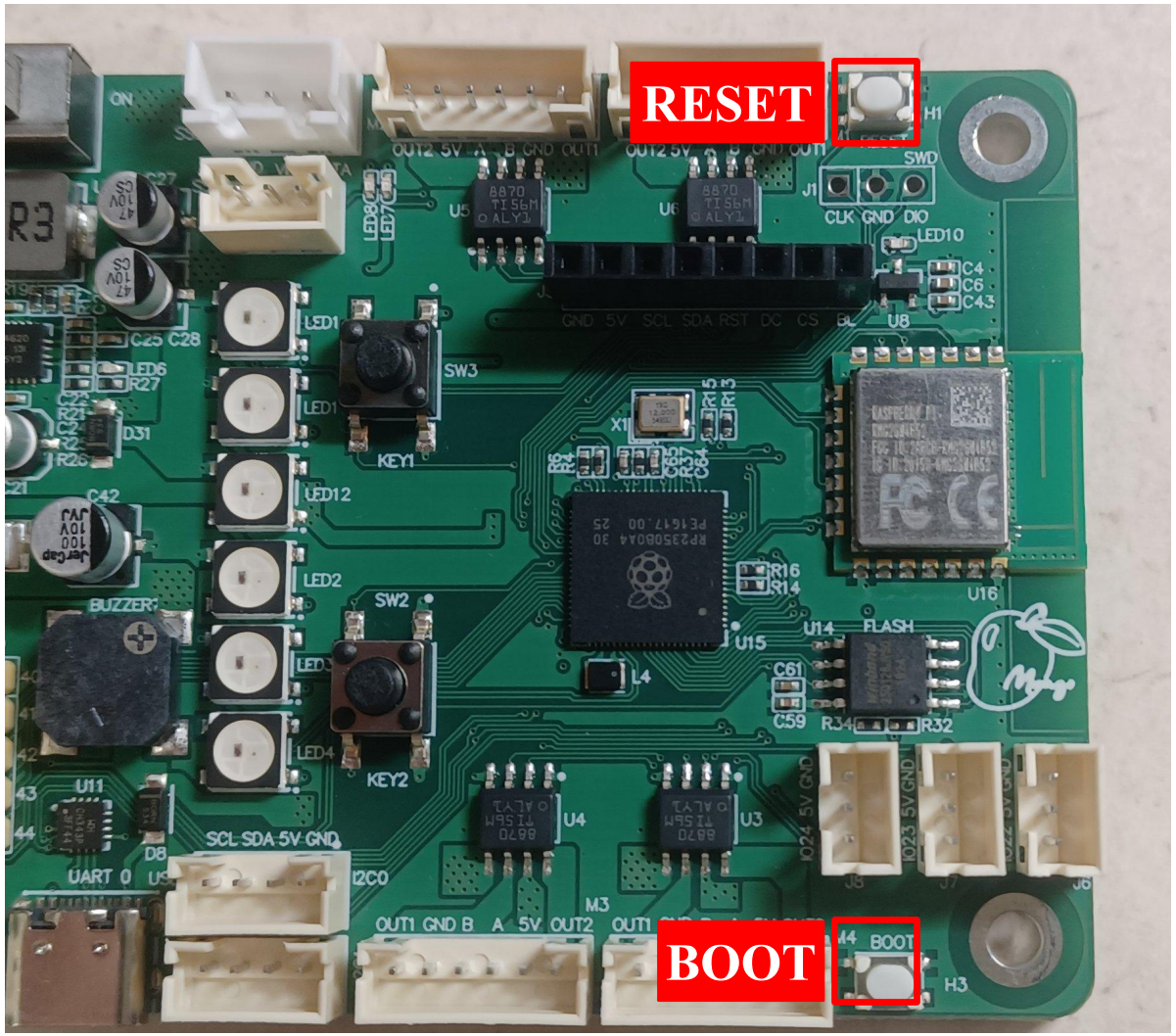


1

Code → Download ZIP

Unzip to your desktop
You'll see morning and firmware

Flash the Firmware



1. **Hold BOOT, press RESET then release BOOT**

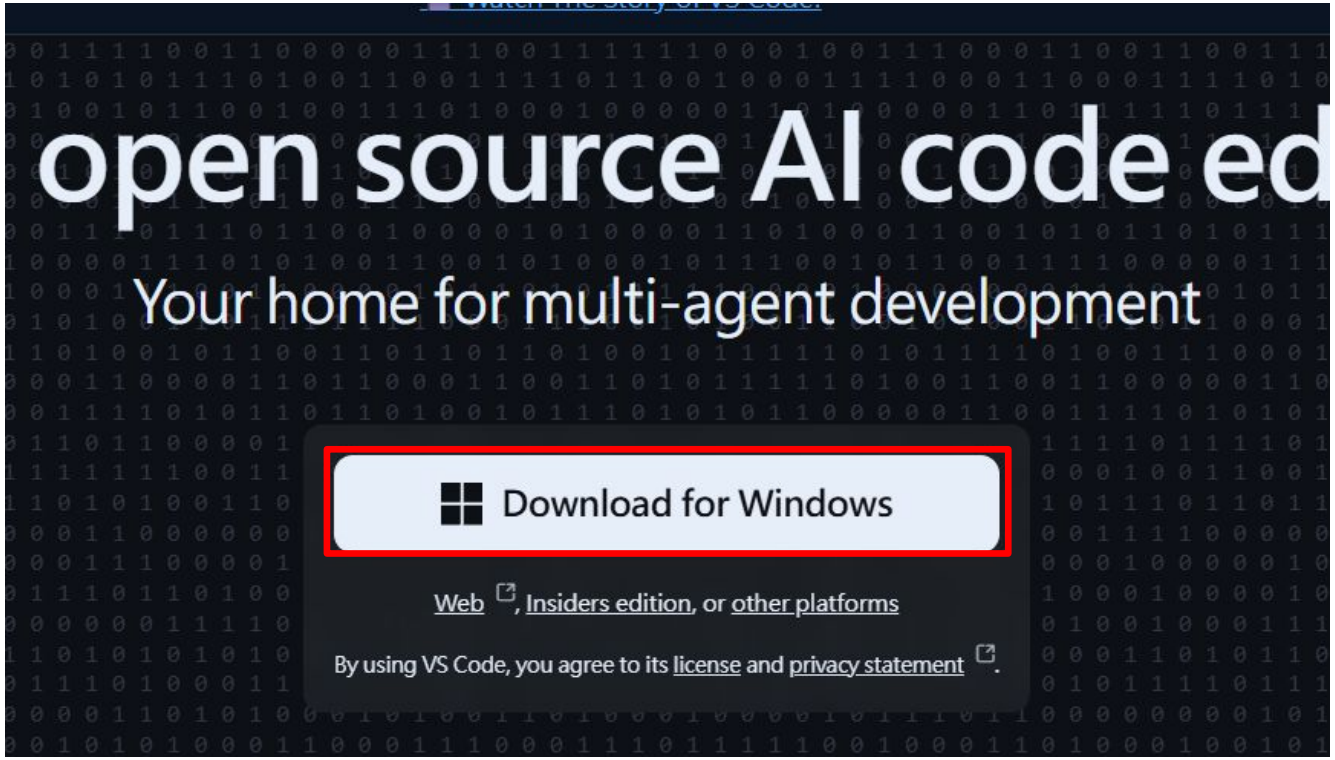
   An RP2350 drive appears

2. **Drag the firmware onto RP2350**

   ```
   firmware/MANGO_BLOCK-v1.29.0.uf2
   ```

3. **Drive disappears = done**

Install VS Code & MicroPico

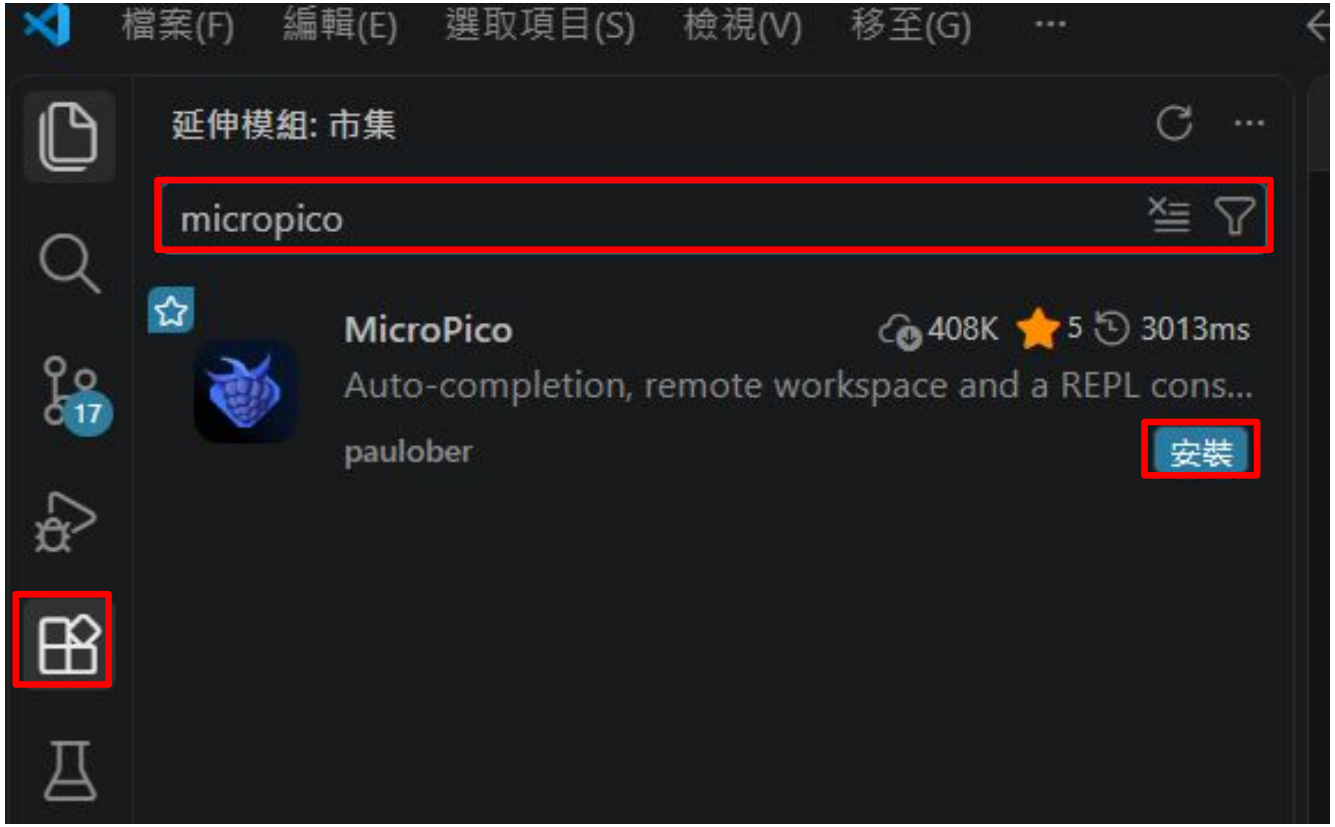


1

Install VS Code

code.visualstudio.com

Install VS Code & MicroPico



2

Search MicroPico
Click Install

Click the Extensions icon on the left (four squares, or Ctrl+Shift+X)
Also install the Python extension it suggests

Install VS Code & MicroPico



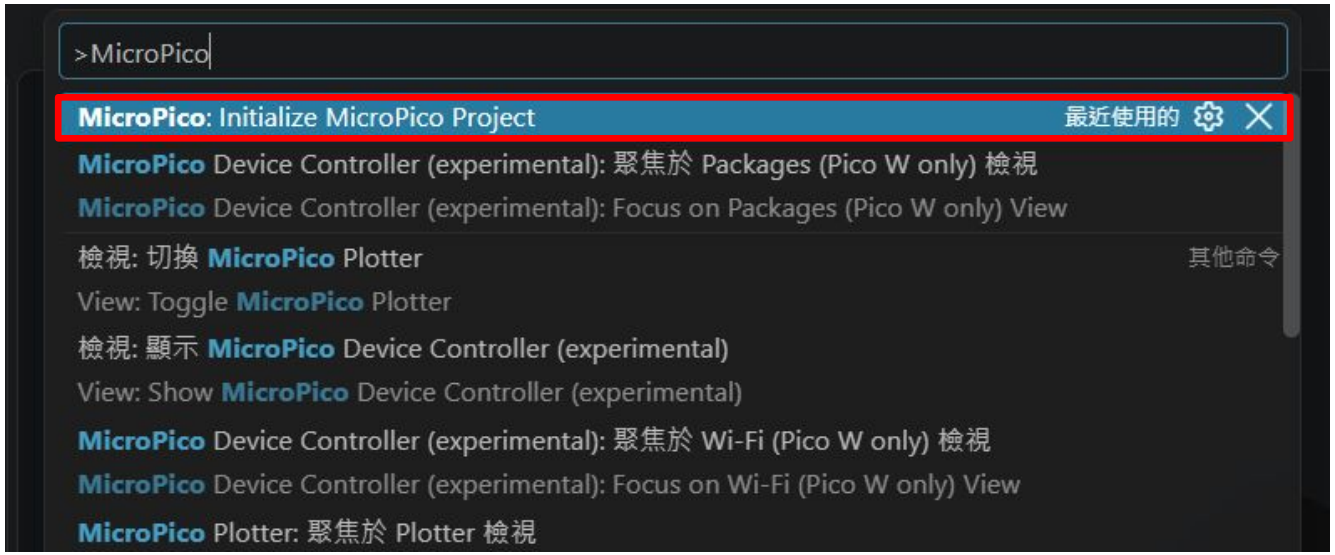
3

Install Python

On python.org, hover over Downloads
Find "Or get the standalone installer for" → click Python 3.15.0
Run the downloaded .exe

Be sure to tick Add python.exe to PATH

Install VS Code & MicroPico



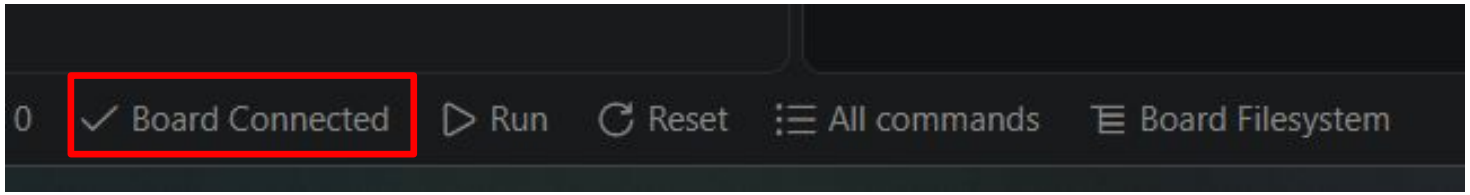
4

Open the folder
Initialize the project

File → Open Folder → course folder
Ctrl+Shift+P → type MicroPico

MicroPico: Initialize MicroPico Project

Connected?



Bottom-left shows

Board Connected

means you're connected

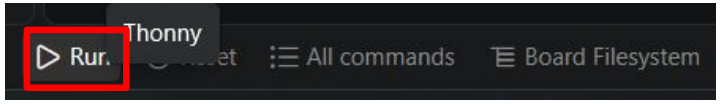
Still Disconnected?

Try first: unplug and replug

Is it in ② USB? ? Try another data-capable cable?

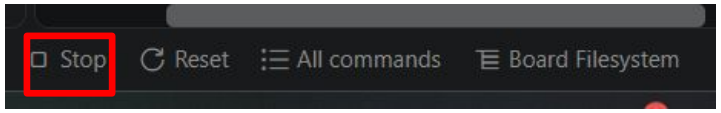
Three Actions

Run



Sends the open file to the board
Doesn't save it on the board

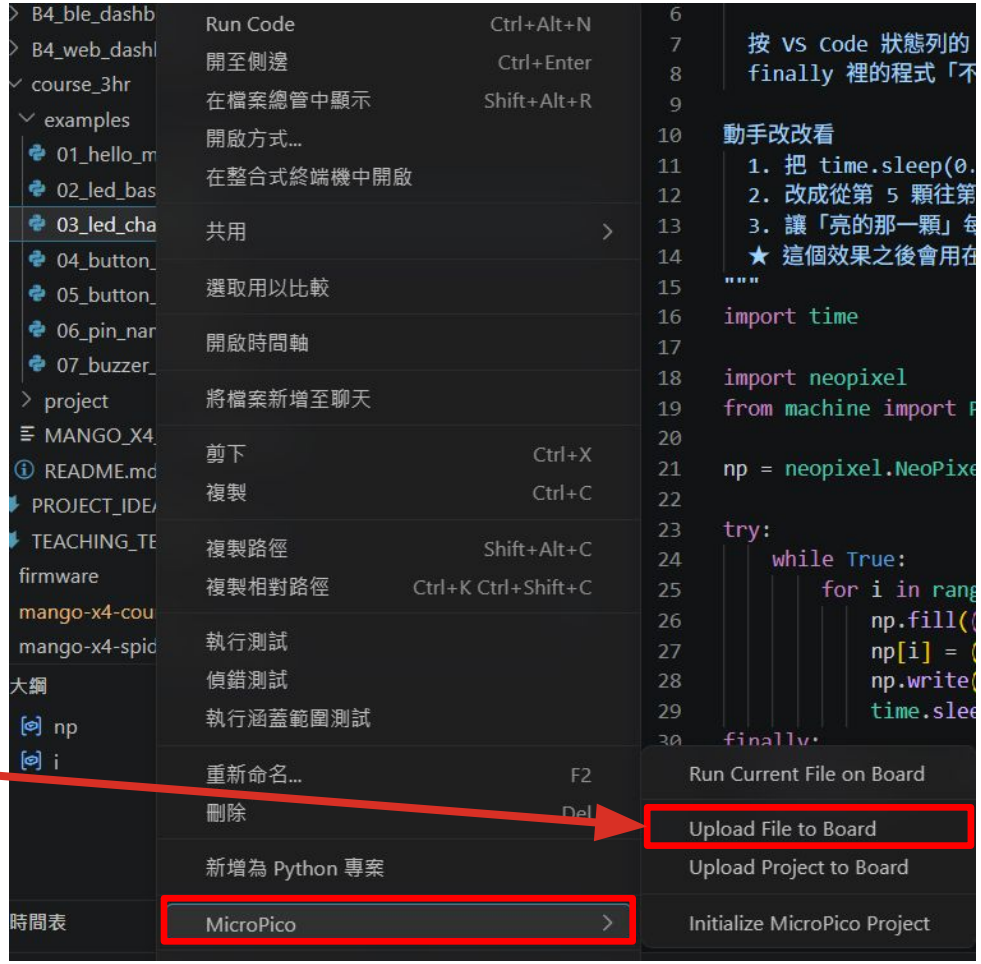
Stop



Stops a while True program
or press Ctrl+C in the terminal

Upload

Right-click file → MicroPico
→ Upload File to Board



Lab 01

Board Roll Call

```
morning/examples/01_hello_mango.py
```

Try it

1. Open the file → click Run
2. 6 LEDs light up and it beeps
3. Done? Change the LED color

Checkpoint Terminal shows "環境 OK！" (setup OK)

`01_hello_mango.py` imports

```
import time

import neopixel
from machine import PWM, Pin
```

These first lines "import modules":
Just run it for now; we'll explain before Lab 02.

Lab 01

20 min

LEDs on, beep, and "環境 OK！" shown

Part 2

LEDs & buttons

Modules · pins · output · input

10:00 – 10:30

Start of a Program: Imports

Top of 01_hello_mango.py

```
import time

import neopixel
from machine import PWM, Pin
```

Python itself only does the basics.
To control the board, import tools first



time

Time: time.sleep(0.1) waits 0.1 s



neopixel

LEDs: controls the 6 WS2812B LEDs



Pin from machine

Pins: find a part on the board by name



PWM from machine

Buzzer: controls pitch and volume

This Morning's Pin Names and Numbers

Name	GPIO	Connected to
WS2812B	45	6 RGB LEDs
BUZZER	46	Buzzer
KEY1	30	Button 1
KEY2	31	Button 2

GPIO = a chip pin your code can control

Name and Number Mean the Same Pin

```
Pin("KEY1")
```

=

```
Pin(30)
```

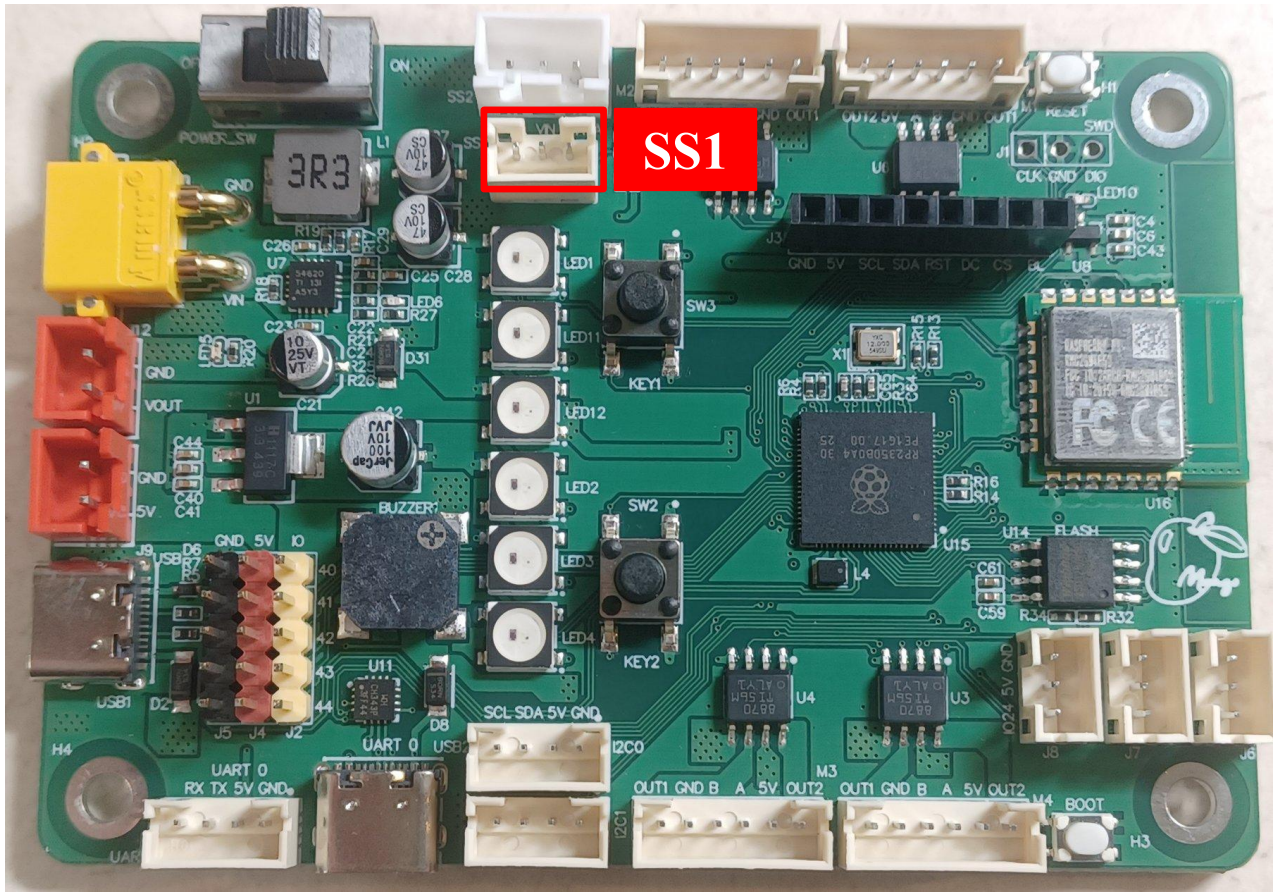
Easy to read, hard to get wrong

You must memorize numbers

The course firmware names every pin; this morning we always use names

All pins on the board: morning/PINS.md

Connector for the Afternoon



SS1

For Hiwonder bus servos

Name	GPIO	Purpose
SS_TX	8	Sends packets
SS_RX	9	Reads replies

One wire, many servos; packet + ID picks which
UART · 115200 baud

Code shows red errors: how to read them

Terminal

```
Traceback (most recent call last):
  File "<stdin>", line 12, in <module>
NameError: name 'led' isn't defined
```

① Read the Error: what kind

NameError = misspelled name
(led should be leds)

② Read line 12: which line

Go to line 12 in VS Code

Last line says	Usually means
`SyntaxError`	Full-width punctuation (, ：()“”) or a missing colon or bracket
`NameError`	Misspelled name, or a variable not created yet
`IndentationError`	Indentation (spaces) not lined up
`TypeError`	Wrong kind of number, e.g. a decimal color; use int()

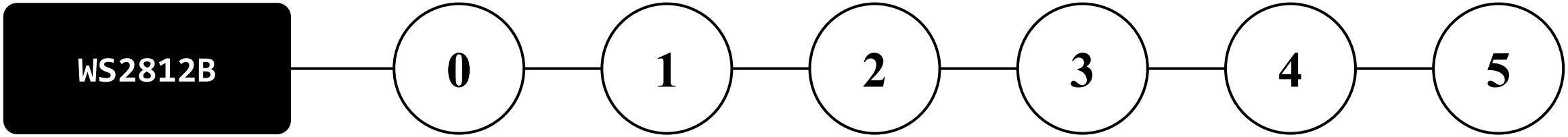
Call Parts by Name

Code	Short name	Used in
`leds = neopixel.NeoPixel(Pin("WS2812B"), 6)` →	**`leds`** 6 RGB LEDs	02, TODO 1, 4
`buzzer = PWM(Pin("BUZZER"))` →	**`buzzer`** Buzzer	04, TODO 3
`button1 = Pin("KEY1", Pin.IN, Pin.PULL_DOWN)` →	**`button1`** Button 1	03, TODO 2

Pin("name") finds the pin, then NeoPixel / PWM turns it into something you can control

Lab 02

6 LEDs on One Pin



Numbered from 0: 0 ~ 5

```
leds = neopixel.NeoPixel(Pin("WS2812B"), 6)
```

Lab 02

Set First, write() Sends

```
leds[0] = RED
```

→

```
leds.write()
```

Only remembers it (RED = (30, 0, 0))
LED doesn't change yet

This line actually sends
the colors to the LEDs

No write(), no change

Lab 02

Control 6 LEDs with a for Loop

02_led.py

```
for number in range(6):
    leds[number] = BLUE
leds.write()
```

number goes through

0	1	2	3	4	5

Set all 6, then write() once

Same color for all? You can write

```
leds.fill(BLUE)
```

Lab 02

Chaser: Draw Frame by Frame

02_led.py

```
# ③ chaser (press Stop to end)
while True:
    for number in range(6):
        leds.fill(OFF)
        leds[number] = GREEN
        leds.write()
        time.sleep(0.1)
```



Turn all off

`leds.fill(OFF)`



Light LED number

`leds[number] = GREEN`



Send

`leds.write()`



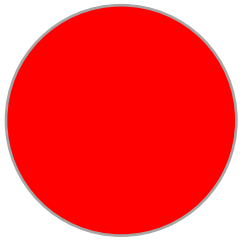
Wait, then next LED

`time.sleep(0.1)`

Animation = swapping pictures quickly
finally at the end: LEDs turn off when you press Stop

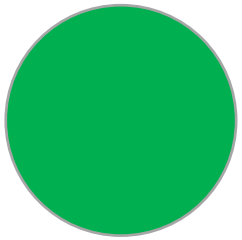
Lab 02

Color = (Red, Green, Blue)



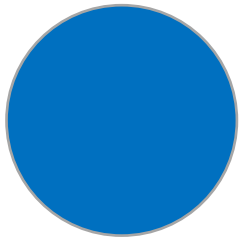
Red

`(30, 0, 0)`



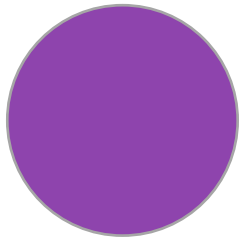
Green

`(0, 30, 0)`



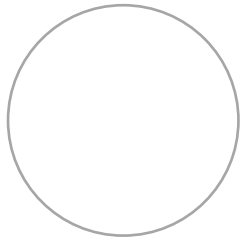
Blue

`(0, 0, 30)`



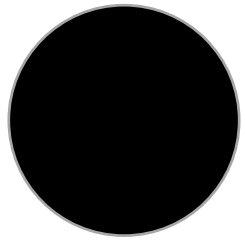
Purple

`(20, 0, 20)`



White

`(30, 30, 30)`



Off

`(0, 0, 0)`

0 ~ 255

Range of each number

30

is bright enough; 255 is blinding

Lab 02

Control the LEDs

```
morning/examples/02_led.py
```

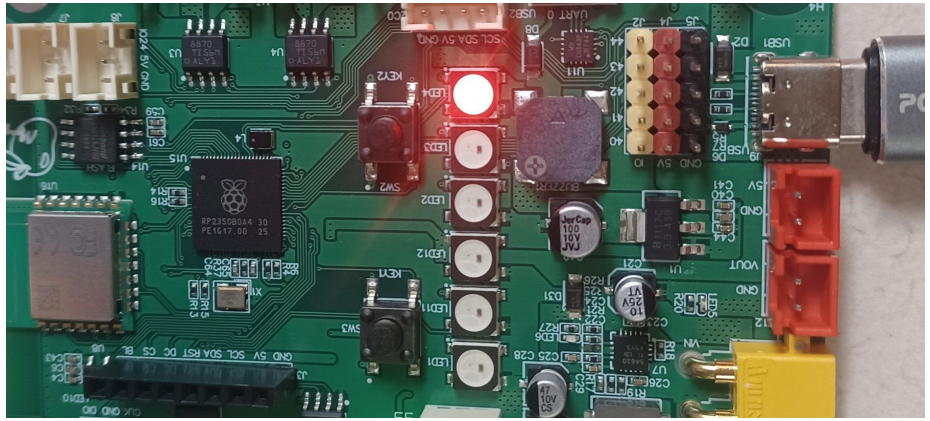
① one → ② all (for loop) → ③ chaser

Try it

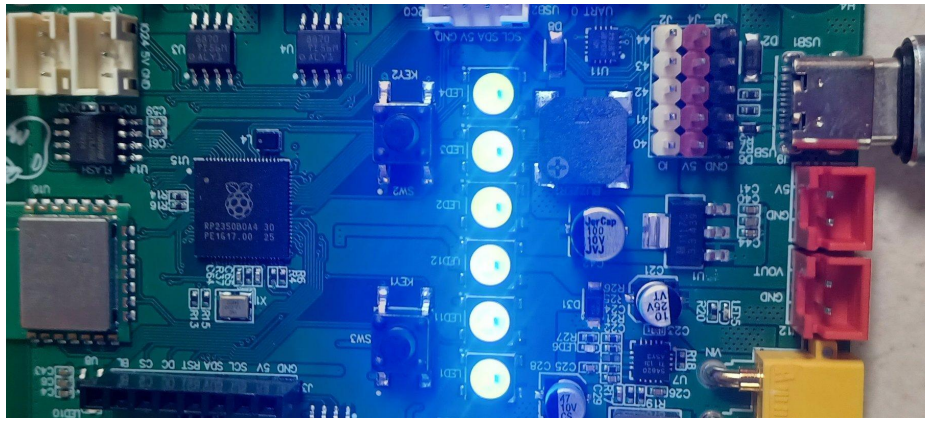
1. Change RED to (0, 0, 30): what color is LED 0?
2. Change the chaser's sleep to 0.3 or 0.03
3. Make a new color PURPLE and use it in the chaser

Checkpoint Light one, set all, and make your own chaser

★ Used again in dashboard TODO 1 and TODO 4



One: LED 0 red



All: 6 LEDs blue

Lab 02

10 min

Lab 03

Read a Button: Set Up, Then Read

Setup

```
button1 = Pin("KEY1", Pin.IN, Pin.PULL_DOWN)
```

↑ Which pin ↑ Input ↑ 0 when not pressed

Read

```
button1.value()
```

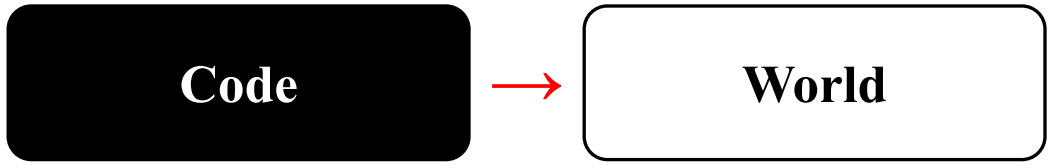
→ **1** Pressed **0** Released

Lab 03

OUTPUT and INPUT

Output

`OUTPUT`



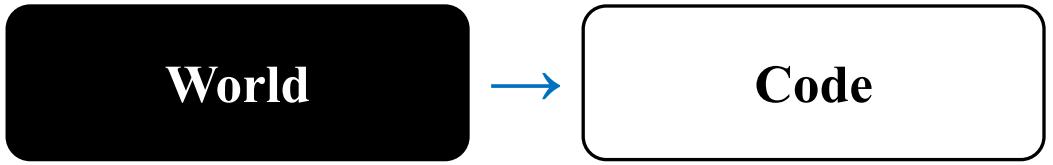
Screen, speaker, lamp

LEDs, buzzer

```
Pin("name", Pin.OUT)
```

Input

`INPUT`



Keyboard, mouse, sensors

Key1, Key2 buttons

```
Pin("name", Pin.IN)
```

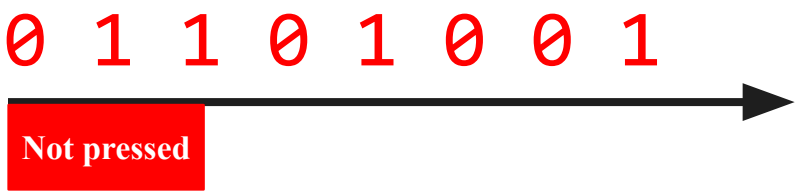
02 and 04 are output: code sends out. From 03 we learn input: code reads in

Lab 03

Why PULL_DOWN?

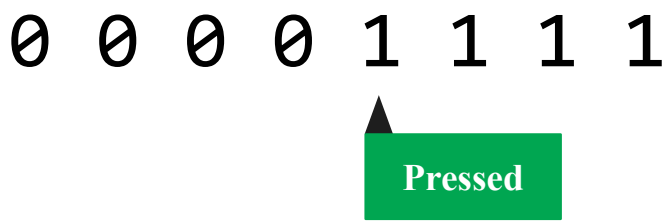
Most boards: without it

When not pressed, the pin is connected to nothing so the value drifts randomly



With Pin.PULL_DOWN

When not pressed, the pin is pulled to 0 so the value is stable



MANGO BLOCK already has pull-down resistors, so it won't drift even without it

Not every board does → make it a habit: add Pin.PULL_DOWN when reading buttons

Lab 03

Read Again and Again, Decide Again and Again

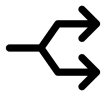
03_button.py

```
while True:
    button1_now = button1.value()
    if button1_now == 1:
        leds.fill(RED)
    else:
        leds.fill(OFF)
    leds.write()
    time.sleep(0.02)
```



Read the button

value() → 0 or 1



Decide

1 = red on, 0 = off



Send

leds.write()



Wait, then repeat

Reads every 0.02 s

It runs so fast it looks like "on while held"

Lab 03

Two Ways to Use a Button



On while held

Like a doorbell: rings while pressed

Button 1: only checks if it's 0 or 1 now

```
if button1_now == 1:
    leds.fill(RED)
```



Press on, press off

Like a room light switch:
press for on, press again for off

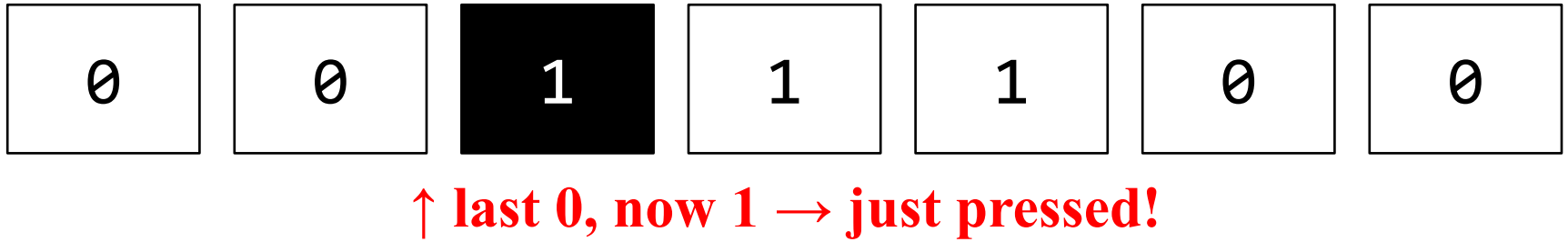
Button 2: must remember "last time"

```
light_is_on = not light_is_on
```

03_button.py has both: button 1 on while held, button 2 toggles

Lab 03

Toggle: Catch the Moment It's Pressed



Holding down doesn't count: it toggles only at the moment it goes from 0 to 1

03_button.py

```
if button2_before == 0 and button2_now == 1:
    light_is_on = not light_is_on       # on ↔ off
button2_before = button2_now            # remember this one for next time
```

Lab 03

What If We Remove time.sleep?

03_button.py

```
while True:
    ...read buttons, pick color...
    leds.write()
    # time.sleep(0.02)
```

Button 2 still toggles

Holding button 1 doesn't light up

LEDs need a short pause to change color

After receiving data they must pause briefly before showing the new color

No sleep: write() after write()

The loop is too fast, the LEDs keep receiving and never get a gap → looks dead

So why does button 2 still work? → next slide

Lab 03

Why Does Button 2 Still Work?

Button 2 prints when it toggles

Printing takes time → gives the LEDs their pause → new color appears

Try it

Keep sleep removed, and add a print under every if

```
if button1_now == 1:
    print("button 1")
```

→ **button 1 lights up too**

Takeaway: leave a little wait in the loop

`time.sleep(0.02)`: the LEDs can update, and button bounce isn't counted many times

Lab 03

On While Held, Press to Toggle

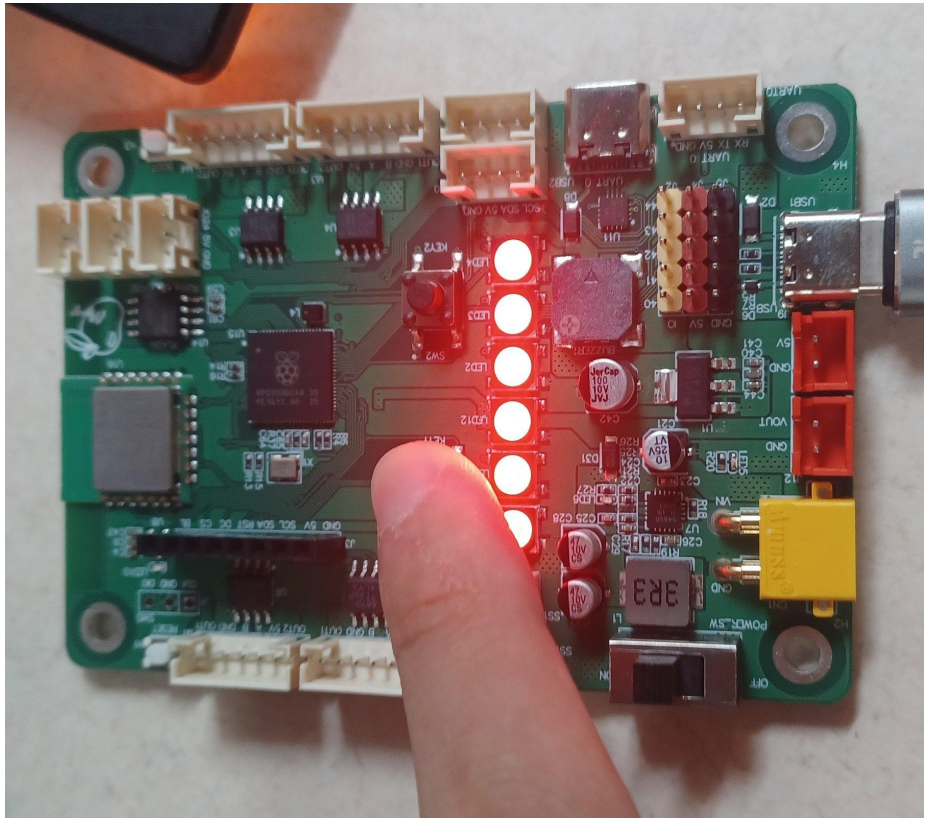
```
morning/examples/03_button.py
```

Button 1: look at "now"
Button 2: catch "last 0, now 1"

Try it

1. Make button 1 light blue (0, 0, 30)
2. Add print(button2_now): how many 1s per press?

Checkpoint Button 1 lights while held, button 2 toggles



Lab 03

10 min

Break

10:30 – 10:40

Part 3

Buzzer

Pitch · volume · play a song

10:40 – 11:00

Lab 04

Lab 01 Already Beeped

01_hello_mango.py

```
buzzer = PWM(Pin("BUZZER"))

buzzer.freq(880)          # pitch
buzzer.duty_u16(3000)     # volume (start sound)
time.sleep(0.2)           # sound for 0.2 s
buzzer.duty_u16(0)        # volume 0 = stop
```

1. **Sound**
   `freq + duty_u16`
2. **Wait**
   `sleep`
3. **Stop**
   `duty_u16(0)`

That's all the buzzer needs: find these 4 lines in 01

No duty_u16(0) → keeps beeping after the program stops

Lab 04

The Buzzer Needs Just Two Functions

`freq()`

Frequency = pitch

Bigger number, higher sound
440 = La

`duty_u16()`

Volume

About 3000 is loud enough; 0 = silent
Max about 32768; beyond that it gets quieter

Lab 04

play(): Wrap a Beep into One Command

04_buzzer_melody.py

```
def play(pitch, seconds):
    if pitch != REST:
        buzzer.freq(pitch)
        buzzer.duty_u16(VOLUME)
    time.sleep(seconds * 0.8)
    buzzer.duty_u16(0)
    time.sleep(seconds * 0.2)
```

```
play(DO, 0.4)   # play Do for 0.4 s
```

1. **`pitch, seconds`**

   Pitch, and how long the note lasts

2. **`pitch != REST`**

   Only sounds if it's not a rest (REST = 0)

3. **`80% sound, 20% silence`**

   so two equal notes stay separate

Write it once, then one line plays a note

Lab 04

The Song Is Data; the Code Just Plays It

Data: the song

```
song = [
    (DO, 0.4), (DO, 0.4),
    (SOL, 0.4), (SOL, 0.4),
    (LA, 0.4), (LA, 0.4), (SOL, 0.8),
]
```

Code: play one by one

```
for pitch, seconds in song:
    play(pitch, seconds)
```

(pitch, seconds)

Pitch 0 = rest

Why 20% silence?

So two equal notes stay apart: Do Do doesn't merge into one long Do

New song = change only the data, not the code

Lab 04

Pitch Chart

Do	Re	Mi	Fa	Sol	La	Si	Do⁺
262	294	330	349	392	440	494	523

The dashboard's piano keys send exactly these numbers

Lab 04

Teacher Demo: A Different Song

teacher/rickroll.py

```
song = [
    (C, S), (D, S), (F, S), (D, S),
    (A, Q), (A, Q), (G, Q * 1.5),
    ...
]
```

Your turn: write your favorite song as (pitch, seconds)

Never Gonna Give You Up

Not one line of play() changed
only the song data

Bonus: each note gets a color, LEDs flash with the music

Lab 04

Twinkle Twinkle

```
morning/examples/04_buzzer_melody.py
```

Try it

1. Finish the second half: Fa Fa Mi Mi Re Re Do
2. Swap in a song you know

Checkpoint Plays your own modified tune



Lab 04

10 min

Part 4

Web dashboard project

The web page is done; you write the board side

11:00 – 11:55

Today's System



Web dashboard → **Bluetooth** → **`mango_ble.py`** → **`dashboard.py`** → **LEDs · buzzer buttons**

Done | **You write** | **From 01 ~ 04**

No Bluetooth theory today; the mango_ble.py library handles it

Click on the web → your function is called

Remember just this one sentence

Labs 01~04 vs. the Dashboard

Labs 01~04: the program runs itself

- Press Run, it goes from first line to last
- while True + sleep sets your own pace
- You call leds.write() yourself

```
while True:
    leds.fill(OFF)
    ...
    leds.write()
    time.sleep(0.1)
```

Dashboard: the web calls you

- Web button → calls your function
- Each function does one small job and ends
- No while True, sleep or write

```
def on_led(led_number,
           red, green, blue):
    # runs when a color is picked
```

What goes inside is still what you learned in 01~04

Same Job, Two Ways

Labs 01~04 (runs itself)

```
buzzer.freq(880)
buzzer.duty_u16(3000)
time.sleep(0.2)
buzzer.duty_u16(0)
```

→

Dashboard (called by the web)

```
def tone_on(pitch):
    buzzer.freq(pitch) ...
def tone_off():
    buzzer.duty_u16(0)
```

01 beep → TODO 3

```
button1_now = button1.value()
```

→

```
def read_keys():
    return key1, key2
```

03 read button → TODO 2

Move the "do it once" lines into a function; drop sleep, the library handles timing

Why No sleep in Your Functions?

The library juggles three jobs at once:



Bluetooth

Always receiving
data from the web



Buttons

Every 0.01 s
asks read_keys



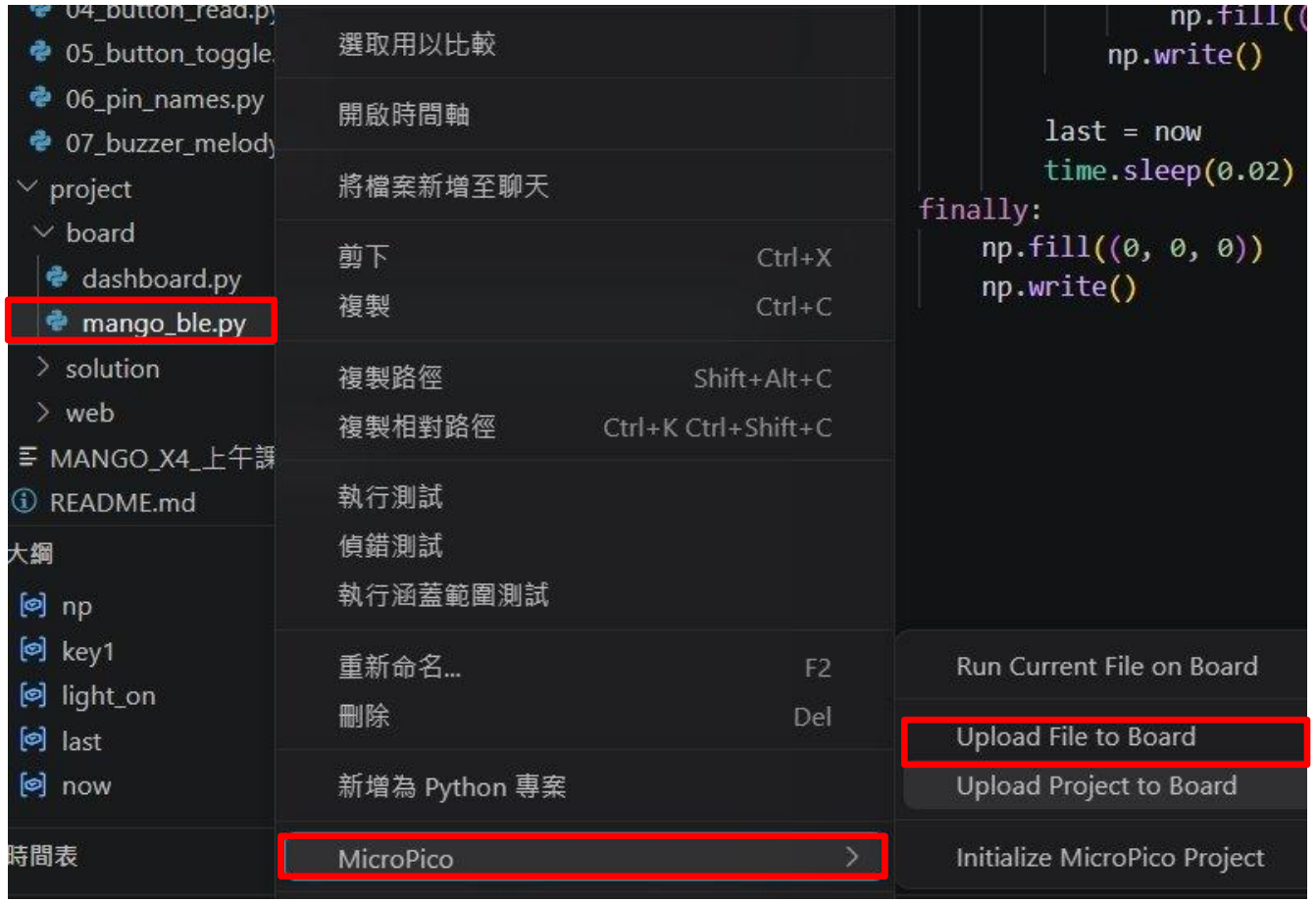
Lights

Every 0.03 s
calls tick

Your function calls time.sleep(1) → for 1 s the library can do nothing
→ buttons freeze, animation stops, web disconnects

If the terminal shows "⚠ … 執行太久" (took too long), that's this warning

Setup Steps

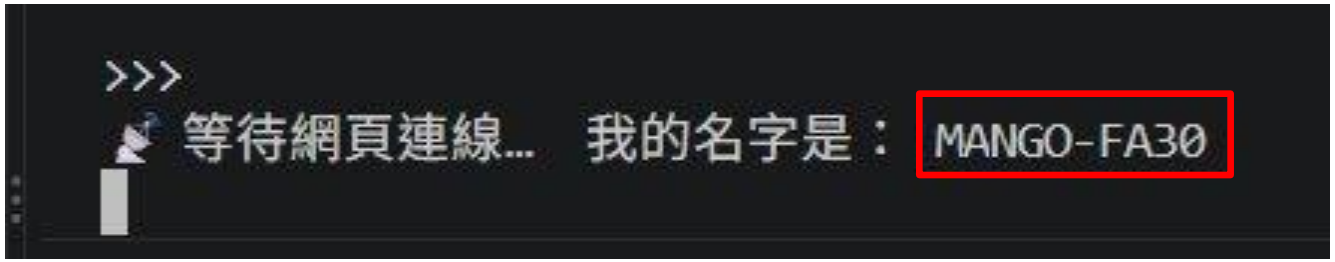


1

Upload the library

Right-click mango_ble.py
→ MicroPico
→ Upload File to Board
Only once

Setup Steps



2

Run dashboard.py
Note your board's name

30 boards in the room
connect to your own

```
MANGO-XXXX
```

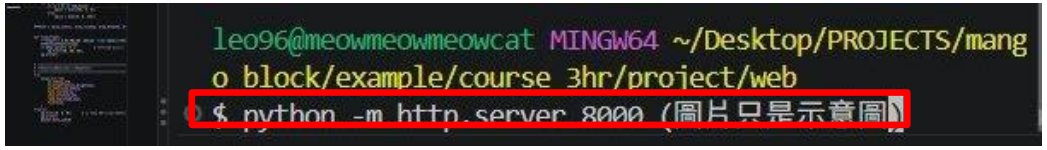
Setup Steps

3 Start the web server



① **Open a new terminal**

Top-right "+" → New Terminal (PowerShell)

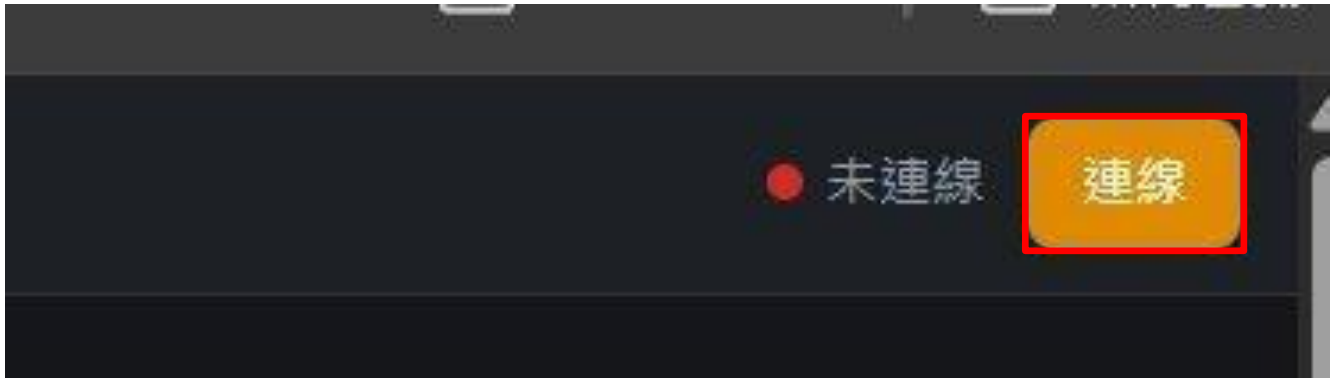


② **Type two commands, Enter after each**

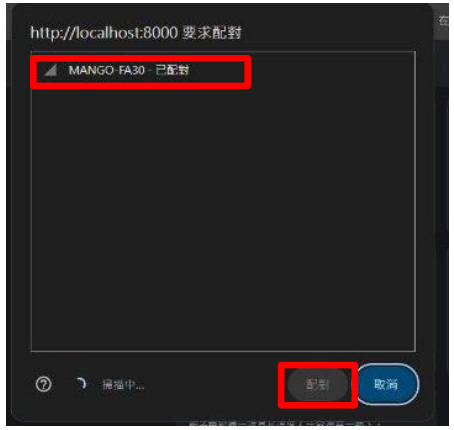
```
cd morning/project/web
python -m http.server 8000
```

Type it in a PowerShell tab, not the MicroPico vREPL
python not found? Use py -m http.server 8000

Setup Steps



① Click "連線" (Connect) at top right



② Pick your MANGO-XXXX → Pair

4

Connect to your board

In Chrome / Edge open
localhost:8000
→ Connect → pick MANGO-XXXX

Checkpoint: "已連線" (connected)

After editing: Stop → Run → reconnect the page

No Web Yet: Call Your Functions Yourself

Bottom of dashboard.py

```
TEST_WITHOUT_WEB = True
```

Set True → Run: like Labs 01~04, you see results right away

```
on_led(255, 255, 0, 0)  # pretend red was picked
run_frames(1)           # pretend 1 s of frames
tone_on(440)            # pretend La was pressed
read_keys()             # print buttons
on_effect(3)            # pretend chaser button
```

1 **Call them yourself**

TEST_WITHOUT_WEB = True
Check every TODO reacts

2 **Let the web call them**

Back to False → Run
Web buttons call the same functions

A function is just "something you can call": it doesn't matter who calls it

TODO 1: on_led

Must **Same as the for loop in 02**

A color was picked: led_number 0 ~ 5 changes one LED, 255 changes all

```
def on_led(led_number, red, green, blue):
    # Pseudocode:
    # if led_number is 255:
    #     for each number:
    #         led_colors[number] = color
    # else:
    #     led_colors[led_number] = color
    # color = (red, green, blue)
```

Test Press "靜態" (static) first, then pick a color, apply, click single dots

Compare with 02

```
for number in range(6):
    leds[number] = BLUE
```

leds → led_colors
BLUE → the color from the web

No write(): tick draws it to the LEDs

TODO 2: read_keys

Must **Same as reading buttons in 03**

Return the state of both buttons (two values)

```
def read_keys():
    # Pseudocode:
    # key1 = value of button1
    # key2 = value of button2
    # return key1, key2
    return 0, 0  # ← change this line
```

Test Hold Key1 / Key2 on the board: squares on the page light up

Compare with 03

```
button1.value()
```

button1 and button2 are already set up for you

Returning two values:
return first, second

TODO 3: tone_on / tone_off

Must **Same as the beep in 01**

Press a piano key on the web → buzzer sounds; the library times it

```
def tone_on(pitch):
    # Pseudocode:
    # set pitch to pitch
    # set volume to VOLUME

def tone_off():
    # set volume to 0

# ⚠ don't write time.sleep
```

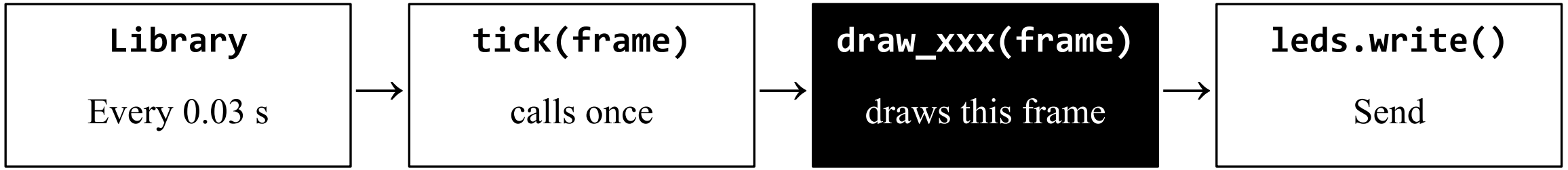
Test Press piano keys; it also beeps on connect and disconnect

Compare with 01

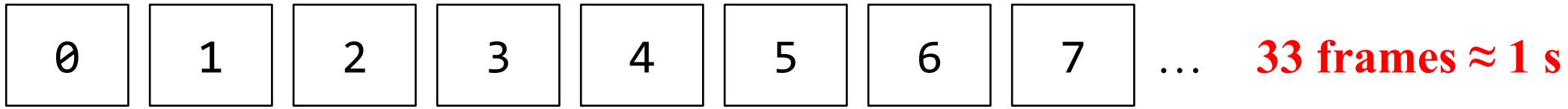
```
buzzer.freq(880)
buzzer.duty_u16(3000)
time.sleep(0.2)
buzzer.duty_u16(0)
```

880 → pitch
3000 → VOLUME
No sleep: the library times it and calls tone_off

What Is frame?



frame = which frame, +1 each time:



33 frames ≈ 1 s

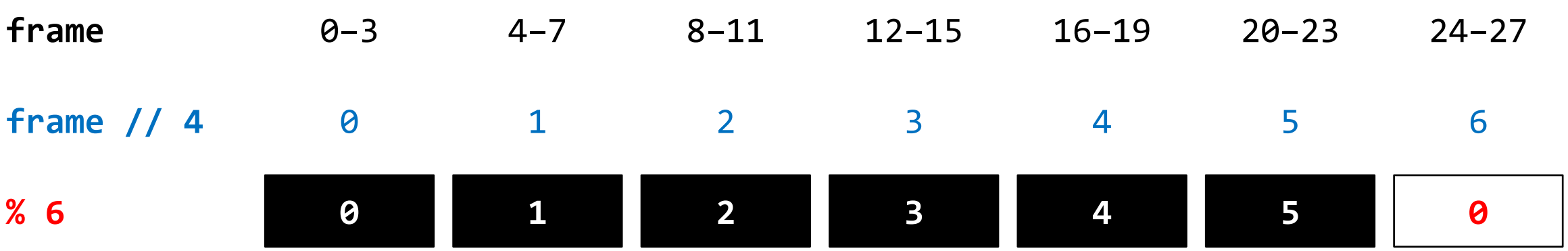
Like a skill cooldown (CD) in games

After using a skill, you wait for the cooldown before using it again. The chaser too: after one LED lights, wait a while before the next.

02: time.sleep(0.1) stops and waits **Dashboard:** can't stop; count frames instead, switch every 4

Whose Turn Is It?

```
position = (frame // 4) % 6
```



// 4: switch every 4 frames

// means "divide, keep the whole number": 0~3 → 0, 4~7 → 1
That's the cooldown: count 4 frames (about 0.12 s), then switch

% 6: only 6 LEDs, back to 0 at 6

% means "remainder after dividing by 6": 6 → 0, 7 → 1
so position always cycles through 0 ~ 5

TODO 4: draw_chase

Must **Almost the same as the chaser in 02**

Called every 0.03 s: just draw "this frame"

```
def draw_chase(frame):
    # Pseudocode:
    position = (frame // 4) % 6
    # for each number:
    #   if number is position:
    #     color of led_colors[position]
    #   else: (0, 0, 0)
```

⚠ No time.sleep / while True

LEDs dark? First set a color with "全部彩燈" (all LEDs)

Test Press "跑馬燈" (chaser) on the web

Compare with 02

```
leds.fill(OFF)
leds[number] = GREEN
leds.write()
time.sleep(0.1)
```

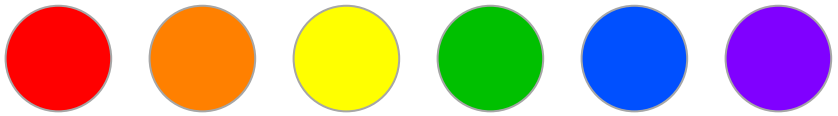
number → position
GREEN → apply_brightness(
led_colors[position])
write and sleep: the library does it

Read It: How the Rainbow Works

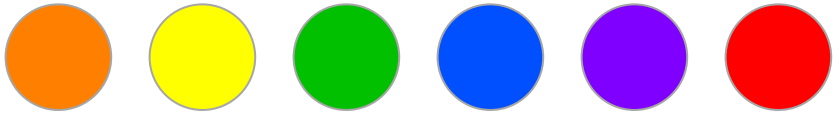
Done **Drawn frame by frame, like the chaser**

Each of the 6 LEDs gets a color; the row shifts every 4 frames

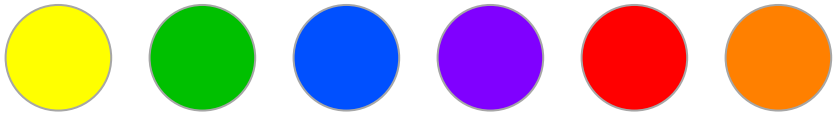
`frame 0–3`



`frame 4–7`



`frame 8–11`



← shifts every 4 frames

Like the chaser: // 4 for cooldown, % 6 to wrap around

RAINBOW is a list of colors

```
RAINBOW = [
  (255, 0, 0),
  (255, 128, 0),
  (255, 255, 0),
  (0, 255, 0),
  (0, 0, 255),
  (128, 0, 255),
]
```

Red, orange, yellow, green, blue, purple
Different colors? Edit these 6 values

Read It: The Rainbow Code

Done **Nearly the same formula as the chaser**

```
def draw_rainbow(frame):
    shift = frame // 4                          # how many steps shifted (cooldown)
    for number in range(6):
        color = RAINBOW[(number + shift) % 6]   # color for LED number
        leds[number] = apply_brightness(color)
```

`shift = frame // 4`

Like the chaser: +1 every 4 frames

`(number + shift) % 6`

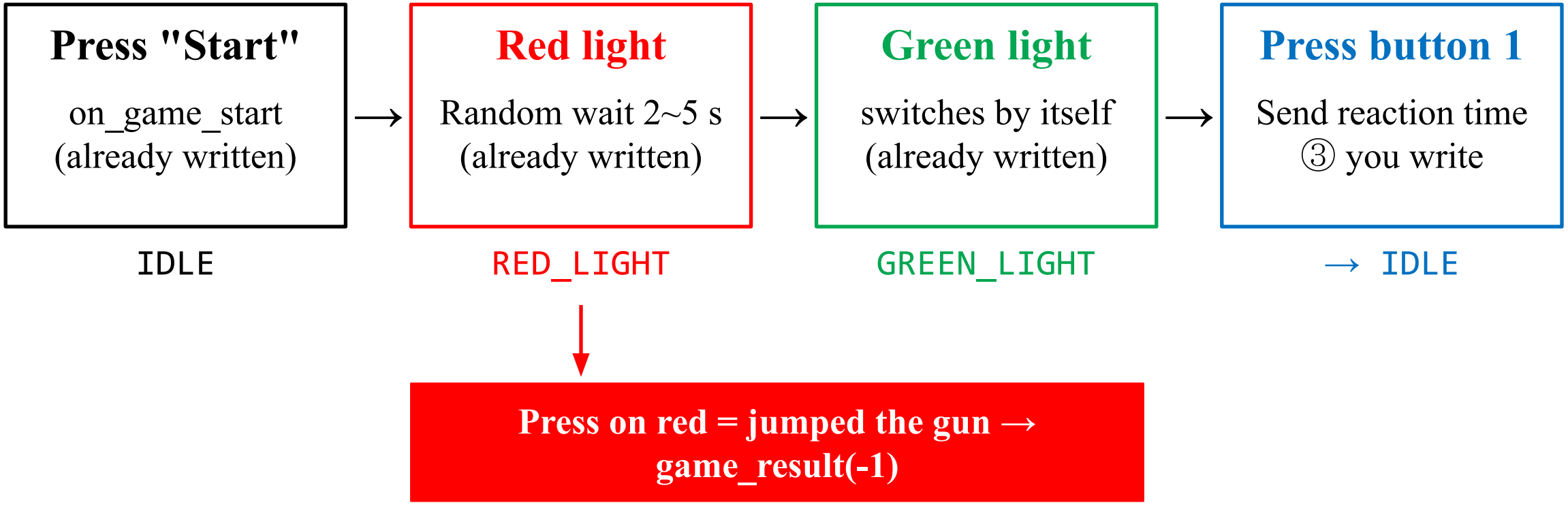
LED number looks shift colors ahead
past 5 it wraps to 0, so colors keep cycling

Try it

Change // 4 to // 2 → spins faster; the breathing effect draw_breathe is also done, read it too

Challenge: Reaction Game (Flow)

Extra **Like button 2 in 03: catch the moment it's pressed**



You write: ② red/green light colors ③ when button 1 is pressed, send the result

Challenge: Reaction Game (Code)

Extra **Goes in draw_game(frame)**

```
# ② if RED_LIGHT: 6 red; else: 6 green

# ③ button1_now = value of button1
#   if last 0 and now 1:
#     if RED_LIGHT: game_result(-1)
#     else: game_result(reaction time)
#     game_state = IDLE
#   button1_before = button1_now
# reaction time = time.ticks_diff(now, green_at_ms)
```

Test On the web, press Start in "小遊戲:反應力" (reaction game)

Bonus: on success mango_ble.beep(1320, 120)

Compare with 03

```
if (before == 0
    and now == 1):
    ...
before = now
```

Remember to update button1_before

ticks_diff(now, green time) = ms since the light turned green

How the Web Page and Code Fit Together

morning/project/	
board/	**Runs on the board**
dashboard.py	**Your functions: all TODOs are here**
mango_ble.py	Board-side Bluetooth library (Upload)
web/	**Served by http.server, opened in the browser**
index.html	Which cards and buttons the page has
app.js	What a click does: mango.write(...)
mango-ble.js	Web-side Bluetooth: talks to mango_ble.py
style.css	Colors, layout

TODOs only need dashboard.py; before going further, read the notes at the top of each file

Free Play: Build It with AI

Button piano

Hold Key1 / Key2
each plays a note

Flashlight

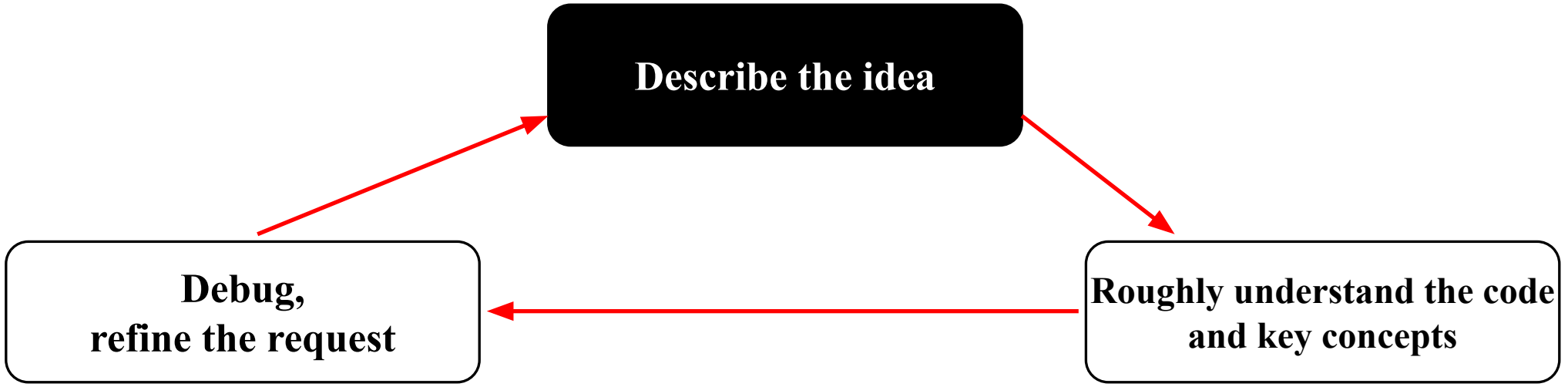
Hold Key2
all 6 LEDs white

Edit the page

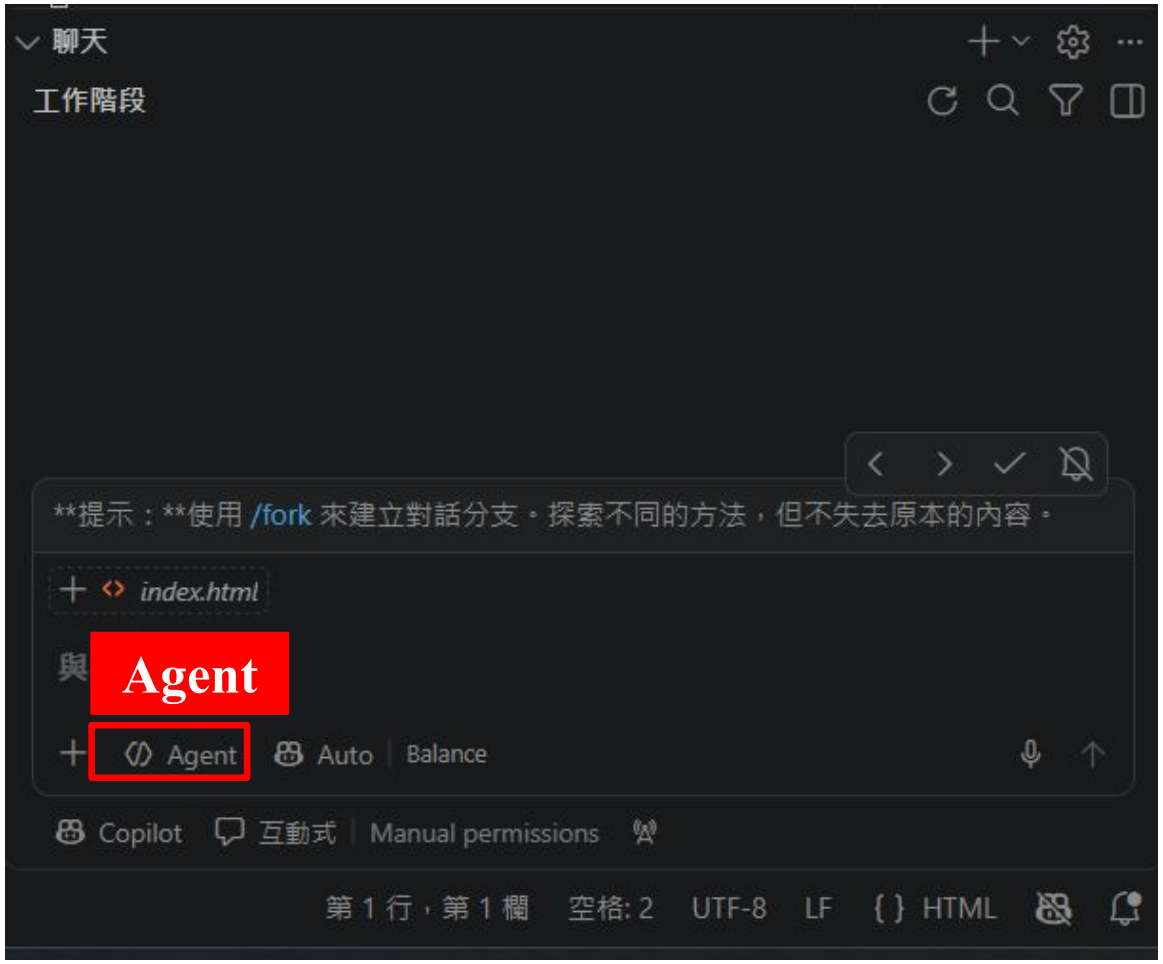
New colors, new layout

New feature

One more web button



VS Code's Built-in AI Agent



1. **Open chat**

   Ctrl+Alt+I, or the chat icon at top right

2. **Choose Agent mode**

   Agent reads files and edits code directly

3. **Describe what you want**

   e.g. add a web button that turns all 6 LEDs white

After changes: Stop → Run → reconnect the page

Web dashboard project

30 min

TODO 1~4 required; then the game challenge, or free play with AI

11:45

Show Time

2 ~ 3 students share their game or free-play project

What We Learned



Output

LEDs write()
Buzzer freq()



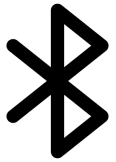
Input

Button value()
catch the press



Pin names

Pin("KEY1")
= Pin(30)



Web control

Click on the web
→ your function is called

Lunch

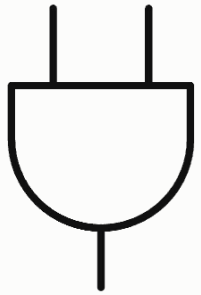
Afternoon: Servo Lab

MANGO BLOCK × Hiwonder bus servos

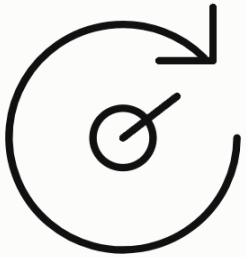
Servo Lab

MANGO BLOCK × Feetech bus servos

Goals



Wire up one



Connect to it
Make it turn



Three at once

Materials



Bus Servo x 3

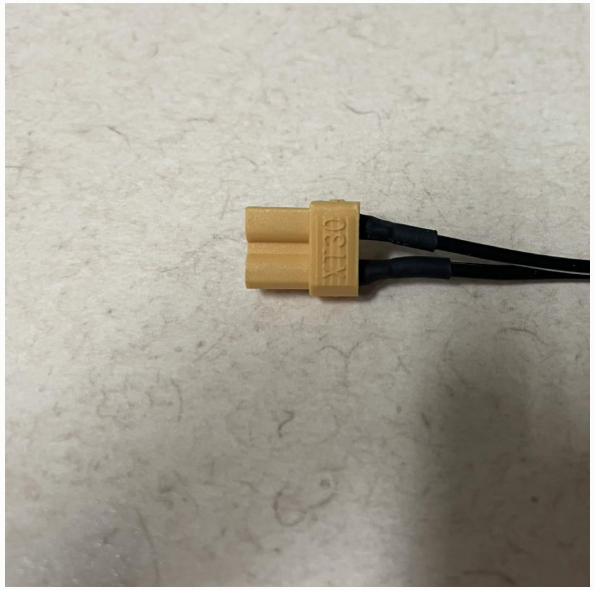


Servo cable x 3

Materials

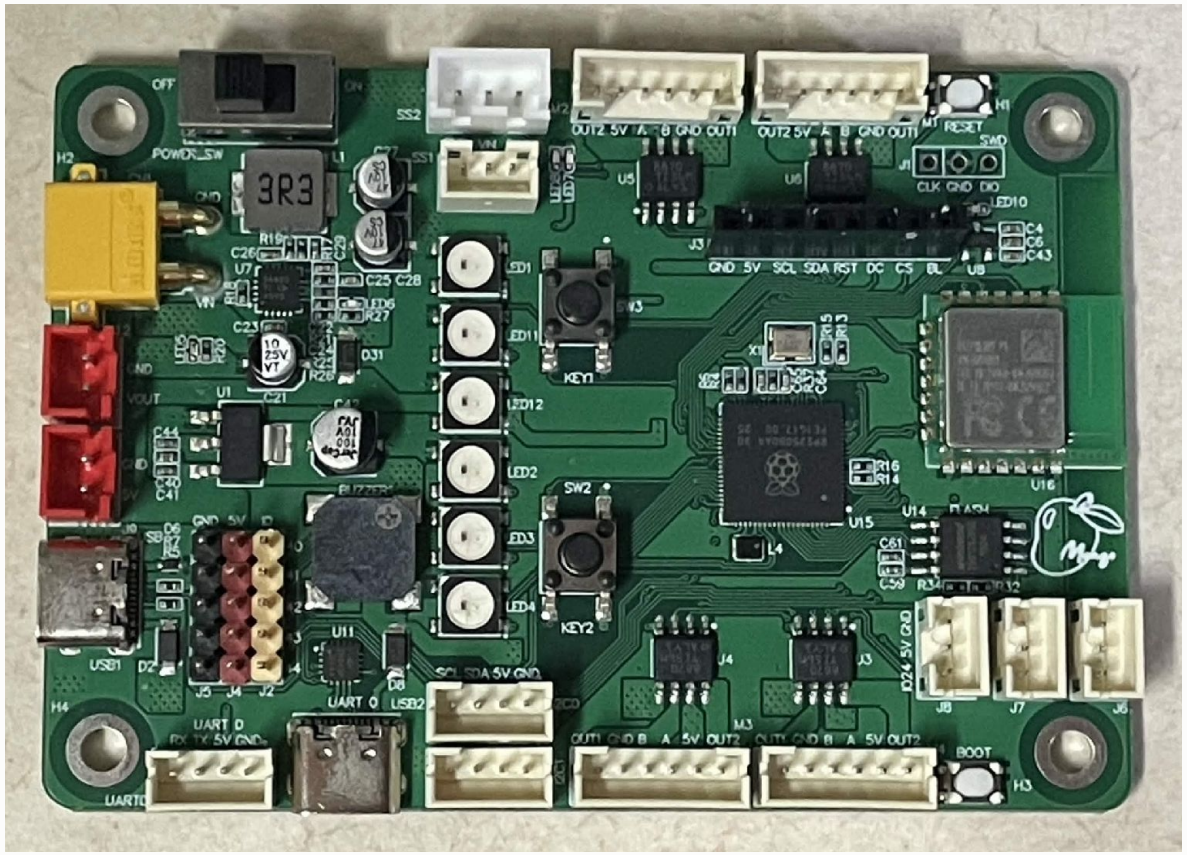


Type-C USB x1



XT30 cable x1

Materials



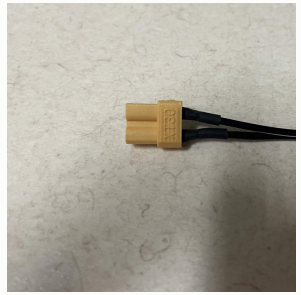
MANGO BLOCK x1

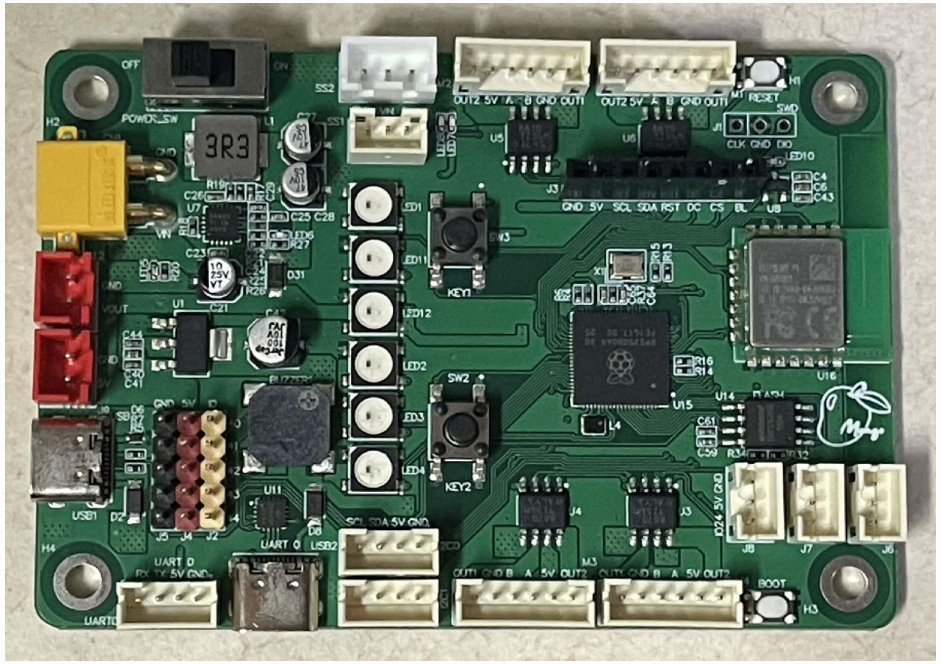
Checklist









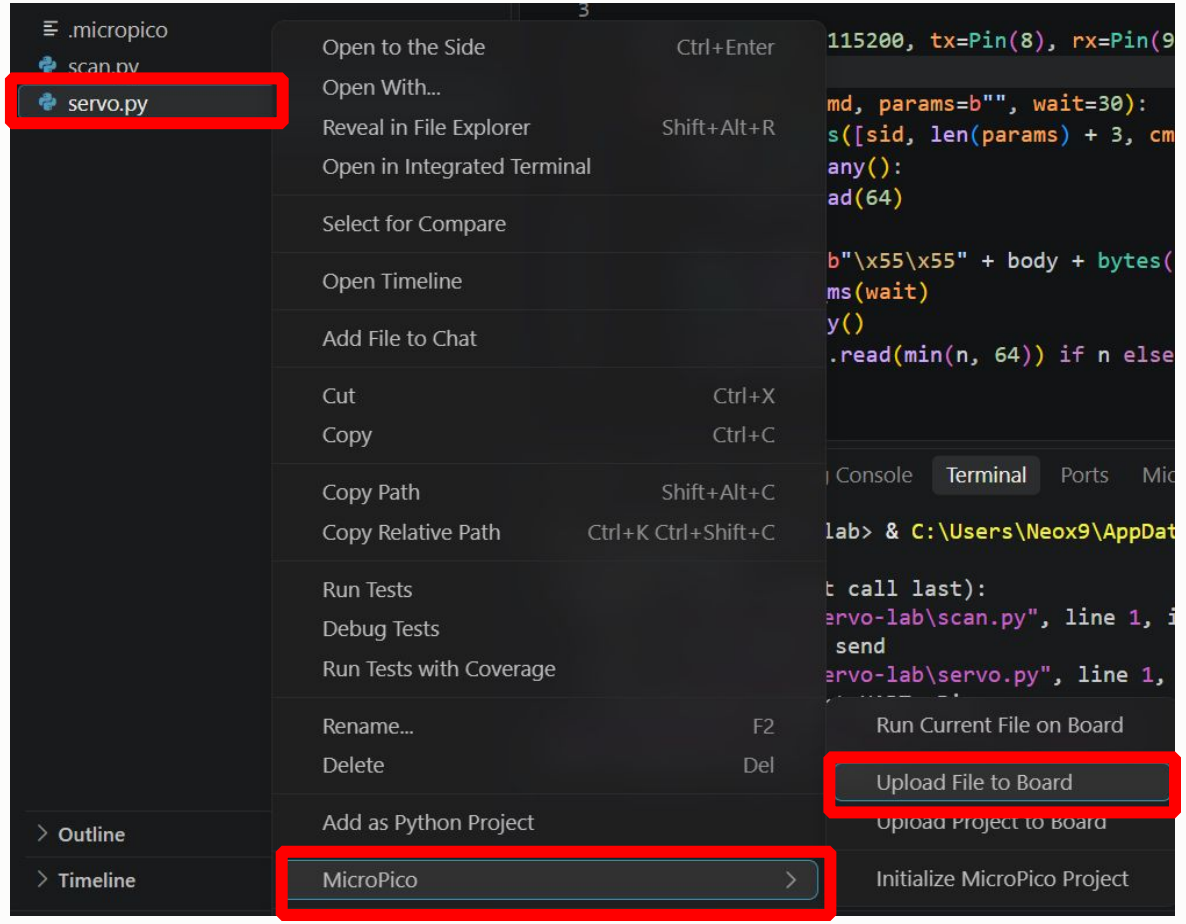


Power off

Before touching any wiring

switch off or unplug the power

Run code from VS Code



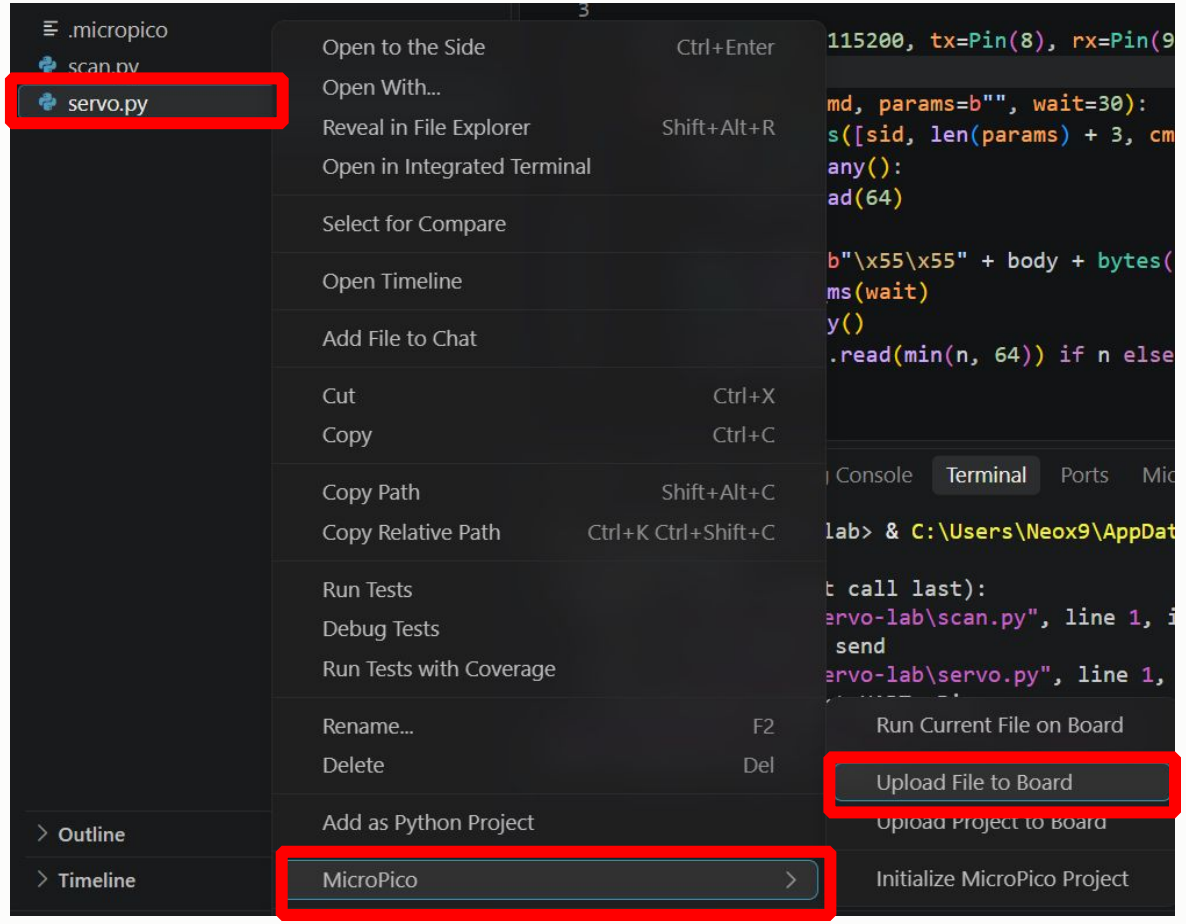
1

Upload servo.py

Right-click the file

→ Upload file to Board

Run code from VS Code



2

Make it beep

Type in the vREPL:

```
from servo import beep
beep(1)
```

Part 1

Servo control & comms

Wire • Read • Talk

Meet the Servo



Hiwonder

HTD-45H

45 kg・cm

High-voltage bus servo

Regular servo (PWM) vs bus servo (Bus Servo)

Regular PWM servo

One pin each

Commands only

Can't read its angle

Bus servo

One wirechains many

Uses packet + ID to pick one

Reports position, volts, temp

Regular servo (PWM) vs bus servo (Bus Servo)





Specs

45 kg · cm
Torque

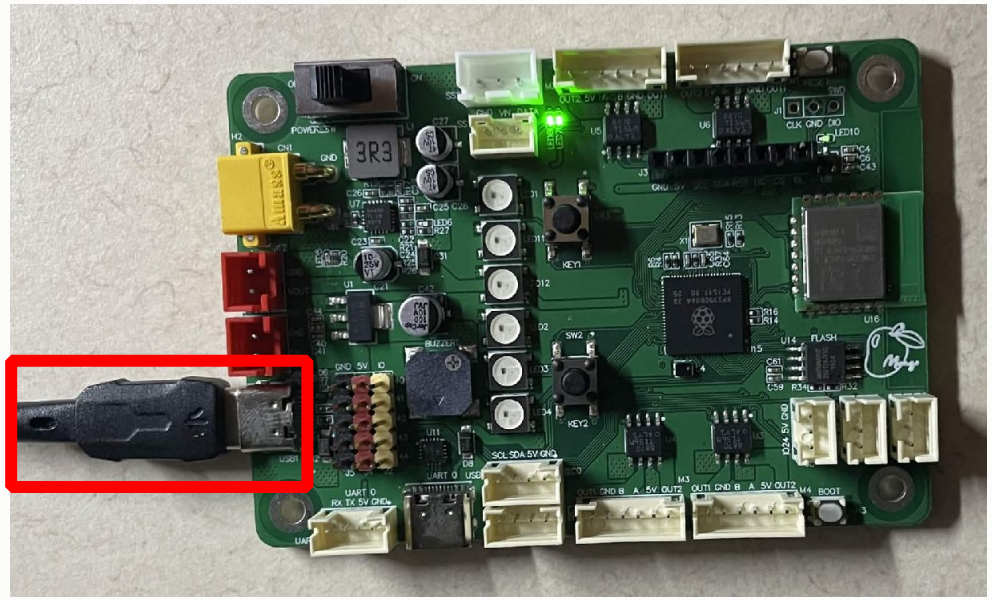
9–12.6 V
Voltage

0–240°
Position 0–1000

ID 0–253
Factory ID is 1

115200
baud · 1-wire half-duplex

Wiring



1

Plug in Type-C

Board on, VS Code connected

Wiring



2

Servo to SS1

Wiring



3

XT30 to 12V power

Keep the switch OFF for now

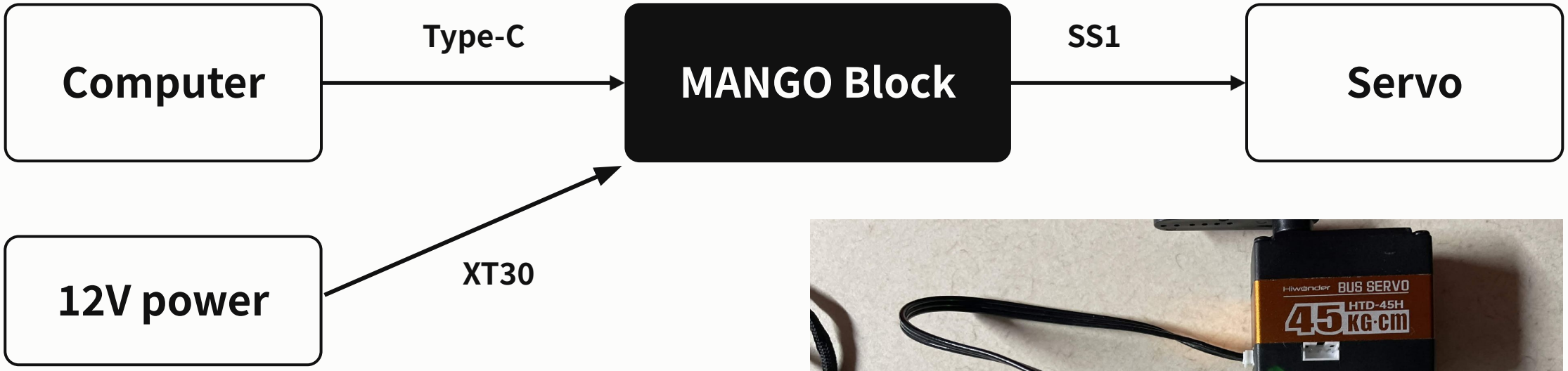
Wiring

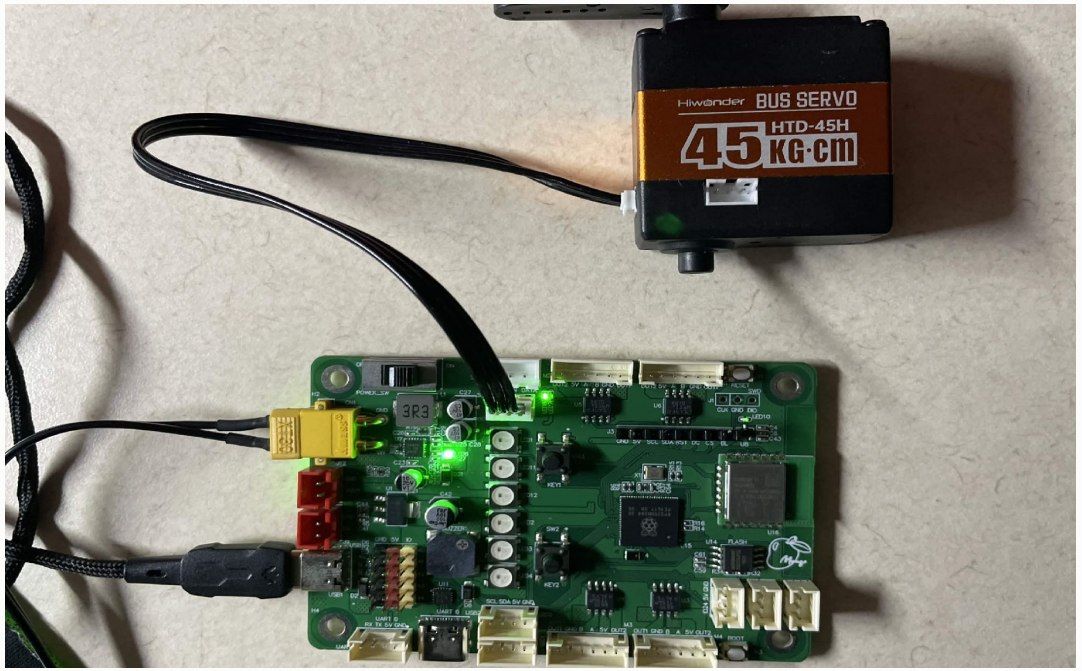


4

Turn POWER_SW ON

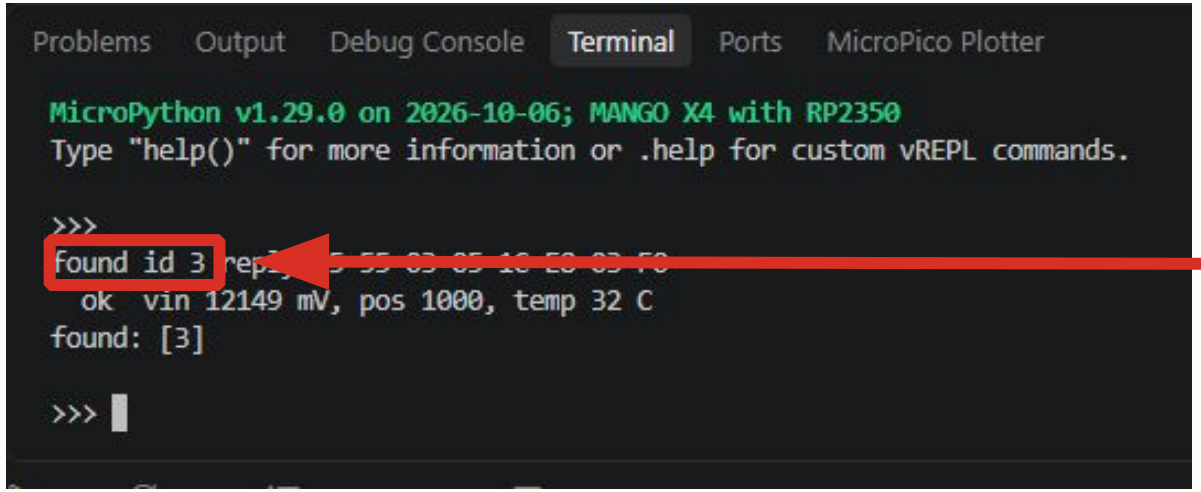
Wiring Map


Computer
Type-C
MANGO Block
SS1
Servo
12V power
XT30


Hiwonder BUS SERVO
45 KG·CM
HTD-45H

Read, Don't Turn

Run scan.py in VS Code • scans IDs 0–253 to find the servo ID

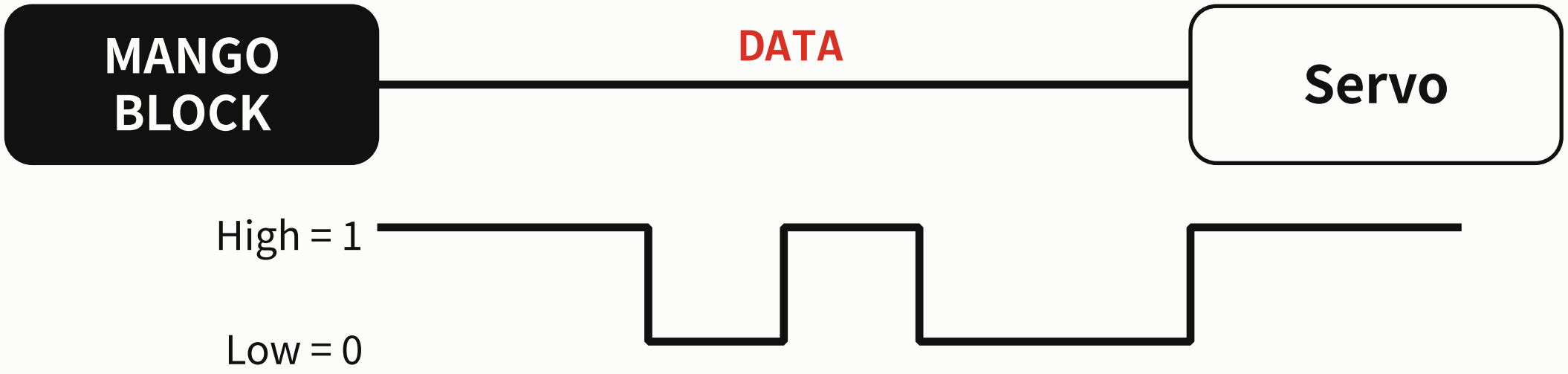


Note this ID

- Found an ID
- About 12 V
- Temp ~30 °C

Communic
ation

How the DATA line sends messages

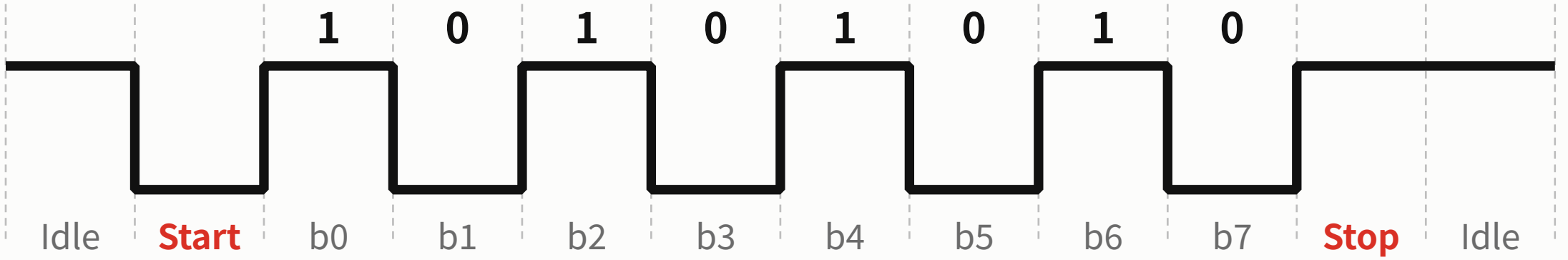


Only one DATA wire carries the data

The line has two states: high (1) and low (0)

When idle, the line stays **high**

How the DATA line sends messages



Sending 0x55 = 0101 0101 (LSB first: 1, 0, 1, 0, 1, 0, 1, 0)

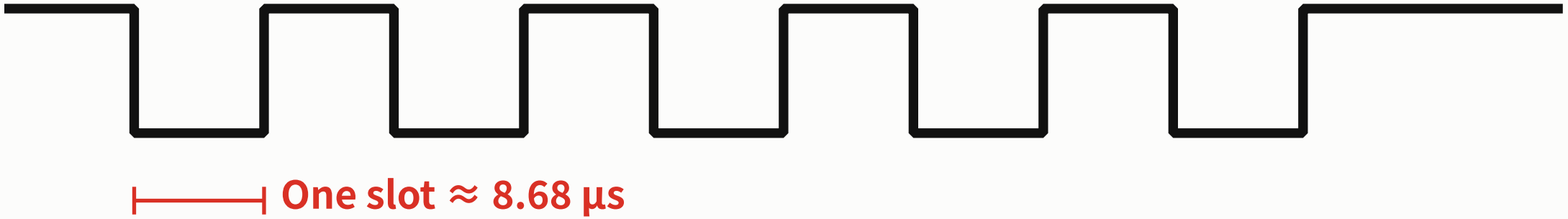
Start bit: goes low for one slot: "here we go"

Data bits: 8 slots = one byte, lowest bit first

Stop bit: back high for one slot: "byte finished"

One byte = 10 slots

How the DATA line sends messages



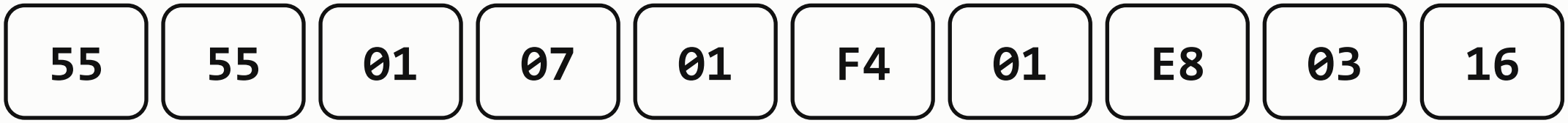
Board 115200 ↔ servo 115200
understood

Board 115200 ↔ servo 9600
wrong slots, garbage

115200 baud: 115200 slots per second, one slot ≈ 8.68 µs

Both sides must use the same speed

How the DATA line sends messages



Packet = a string of bytes

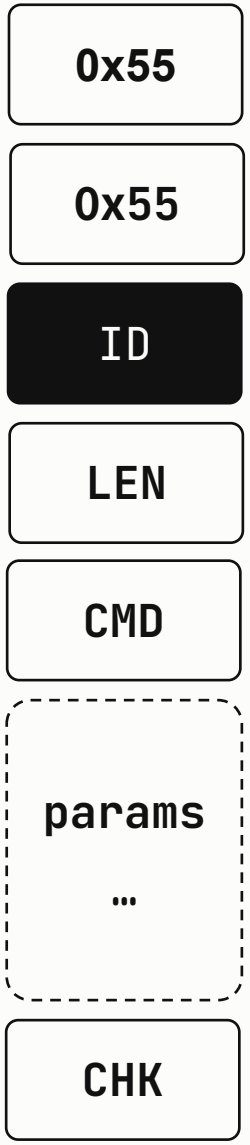
Each byte is "start + 8 bits + stop"

The packet for move(1, 500, 1000) is 10 bytes = 100 slots

115200 baud at about 0.87 ms it's sent

Packet

What a Packet Looks Like



`Header (two 0x55)`

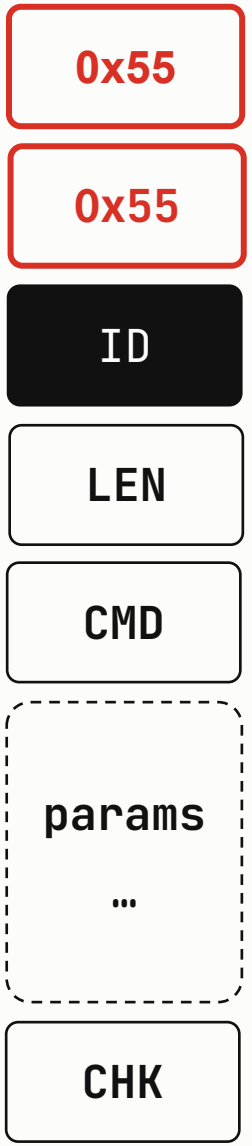
Servo ID - ID of the target servo

LENGTH = bytes in ID + LEN + CMD + params

COMMAND - what the servo should do

PARAMETERS - params (some commands have none)

`CHK = ~(ID + LEN + CMD + params) & 0xFF`



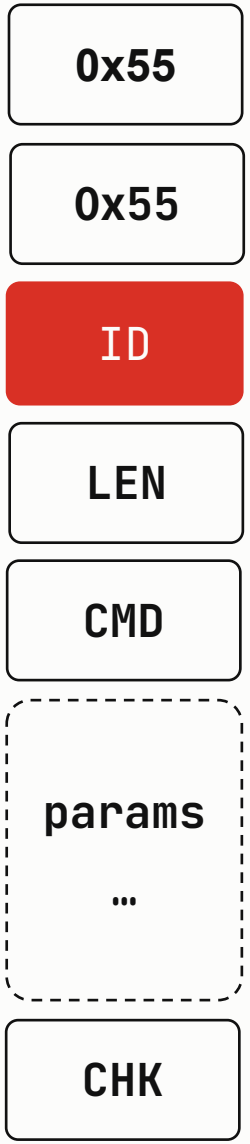
Header

Always two 0x55: "the packet starts here"

0x55 = 0101 0101

Alternating 0s and 1s help the receiver sync timing

Two 55s in a row are rare, so it's hard to mistake



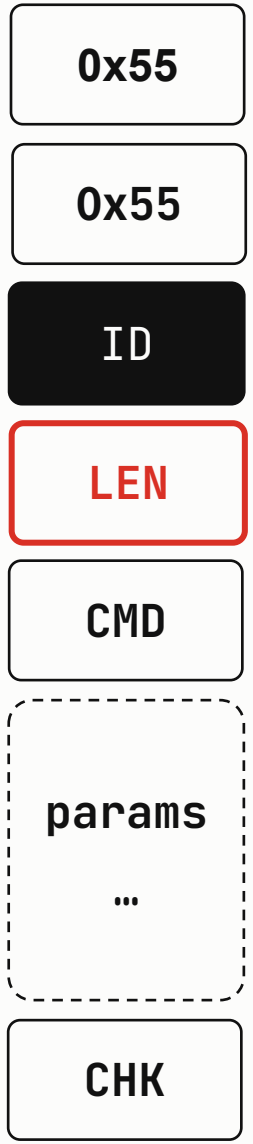
SERVO ID

ID 0–253

Chooses which servo runs the command

Every servo hears it; only the one with the matching ID moves and replies

254 (0xFE) is broadcast



LENGTH

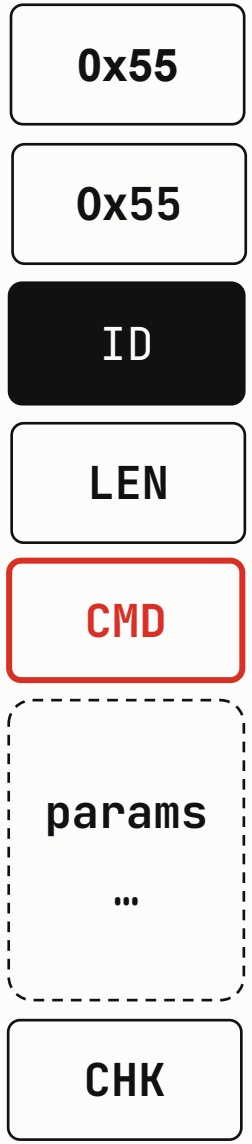
LEN = number of params + 3(ID, LEN, CMD)

Tells the receiver how many more bytes to read

Read voltage 0 params → **LEN 3**

Change ID 1 param → **LEN 4**

Move 4 params → **LEN 7**



COMMAND

CMD tells the servo what to do

Read: 26 temperature, 27 voltage, 28 position → servo replies

Write: 1 move, 13 change ID → no reply

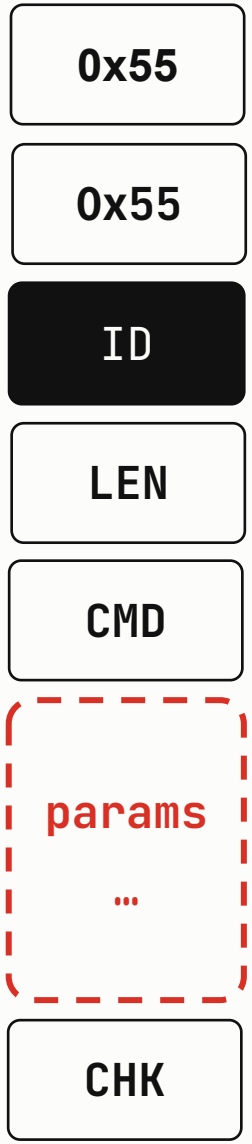
The reply's CMD matches what you sent, so you can check it

Voltage • CMD 27
returns mV

Position • CMD 28
returns 0–1000

Temp • CMD 26
returns °C

Angle limits • CMD 21
returns min/max



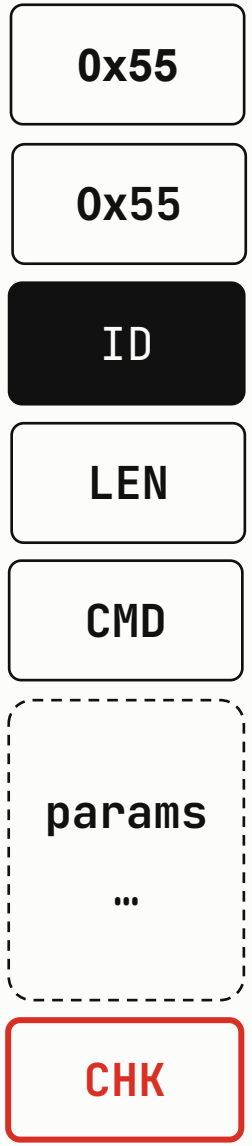
Params PARAMETERS

Each command needs different params; reads usually need none

Numbers over 255 are split into two bytes, low byte first

500 = 0x01F4 → F4 01

1000 = 0x03E8 → E8 03



CHECK

```
CHK = ~(ID + LEN + CMD + params) & 0xFF
```

1. Add: 01+07+01+F4+01+E8+03 = 0x1E9
2. Keep the lowest byte: 0xE9
3. Flip every bit: ~0xE9 = 0x16

The receiver computes it again;if it doesn't match, drop it

Send the Packet

servo.py

```
from machine import UART, Pin, PWM
import time

uart = UART(1, 115200, tx=Pin(8), rx=Pin(9))

def send(sid, cmd, params=b"", wait=30) -> bytes:
    body = bytes([sid, len(params) + 3, cmd]) + params

    while uart.any():
        uart.read(64)

    uart.write(b"\x55\x55" + body + bytes([~sum(body) & 0xFF]))

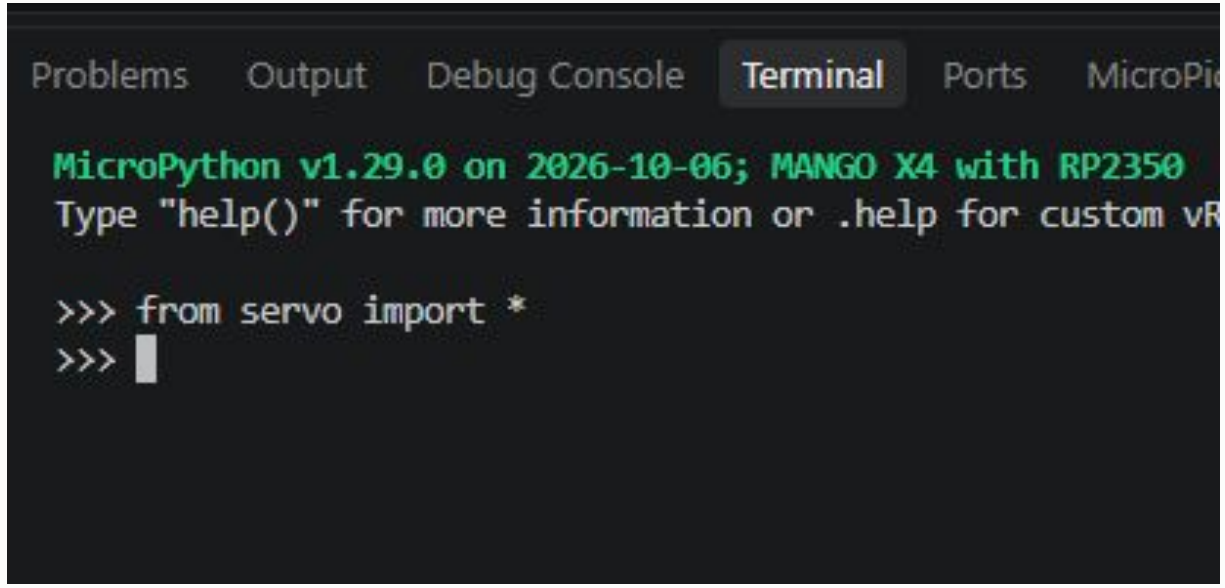
    time.sleep_ms(wait)
    n = uart.any()
    return (uart.read(min(n, 64)) or b"") if n else b""
```

Make it turn

After uploading servo.py you can use these four commands

`move(id, position, ms)`	Turn to a position, 0–1000
`pos(id)`	Read position
`vin(id)`	Read voltage in mV
`beep(n)`	Beep n times

Make it turn



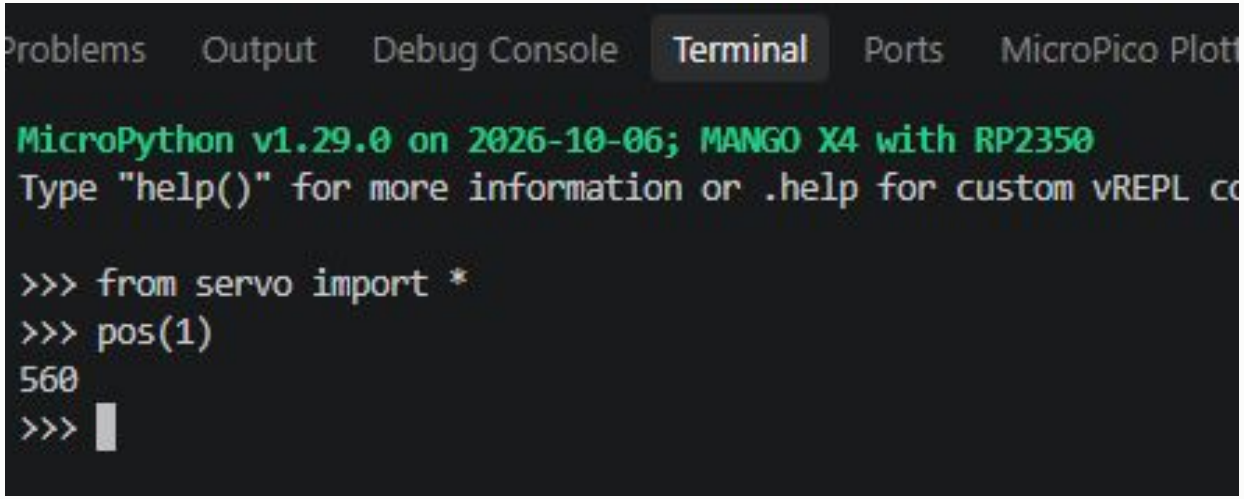
1

Import the

```
>>> from servo import *
```

Type it again after every board restart

Make it turn



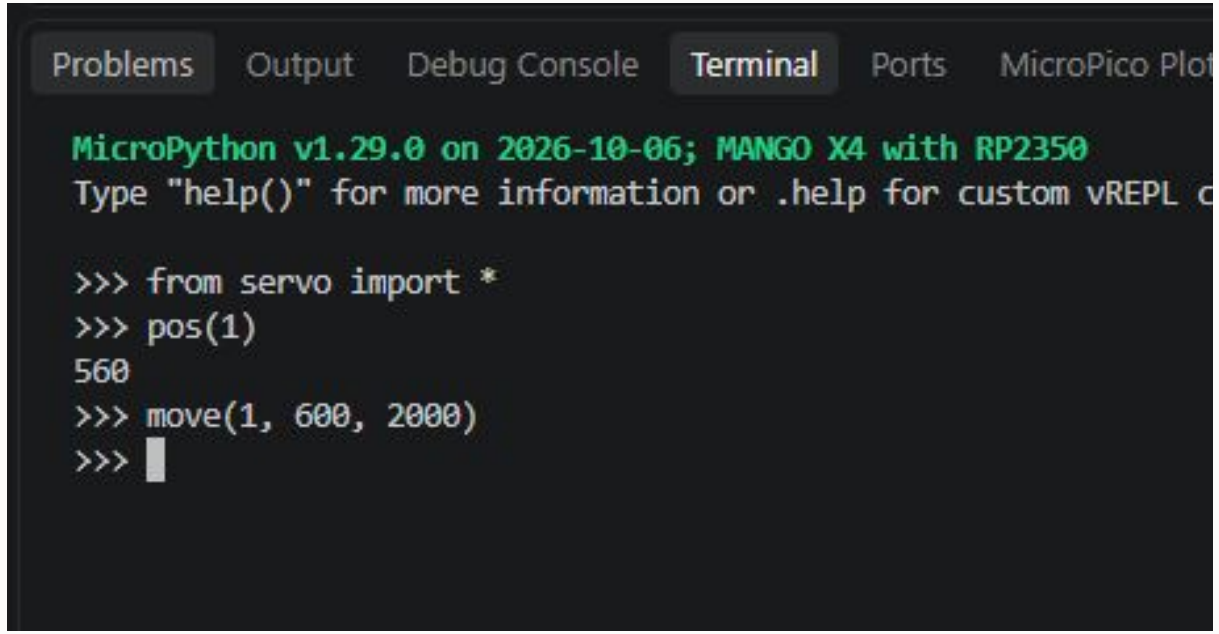
2

Read position

```
>>> pos(1)
560
```

First see where it stopped

Make it turn



3

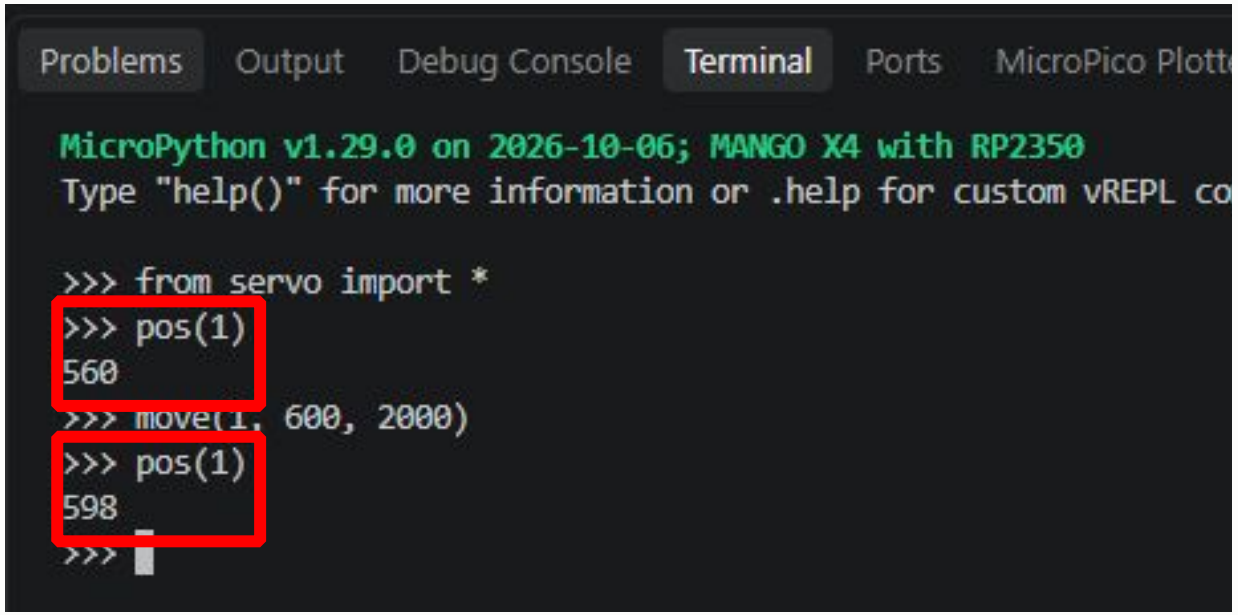
Turn to 600

```
>>> move(1, 600, 2000)
```

Reach 600 in 2 s

Hold the servo in place

Make it turn



4

Check it arrived

```
>>> pos(1)
598
```

A tiny difference is normal

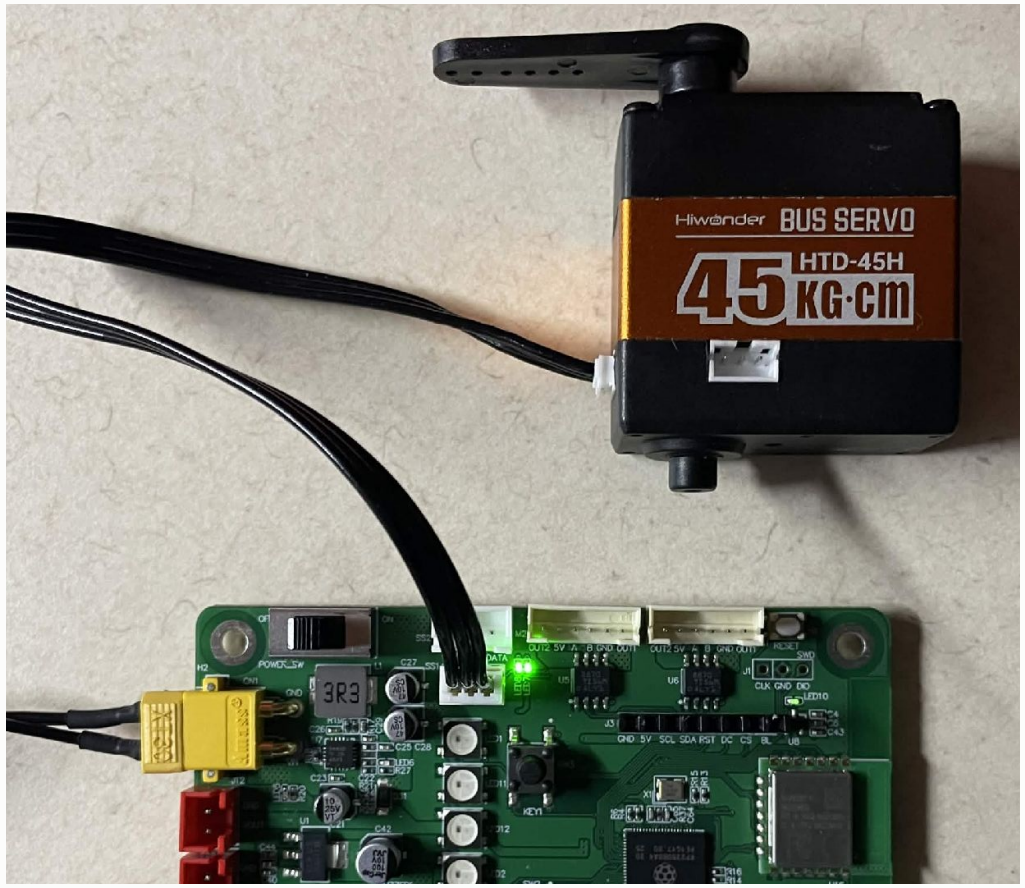
Part 2

Many Servos

Chain · IDs · Turns

Set ID

Change Servo ID

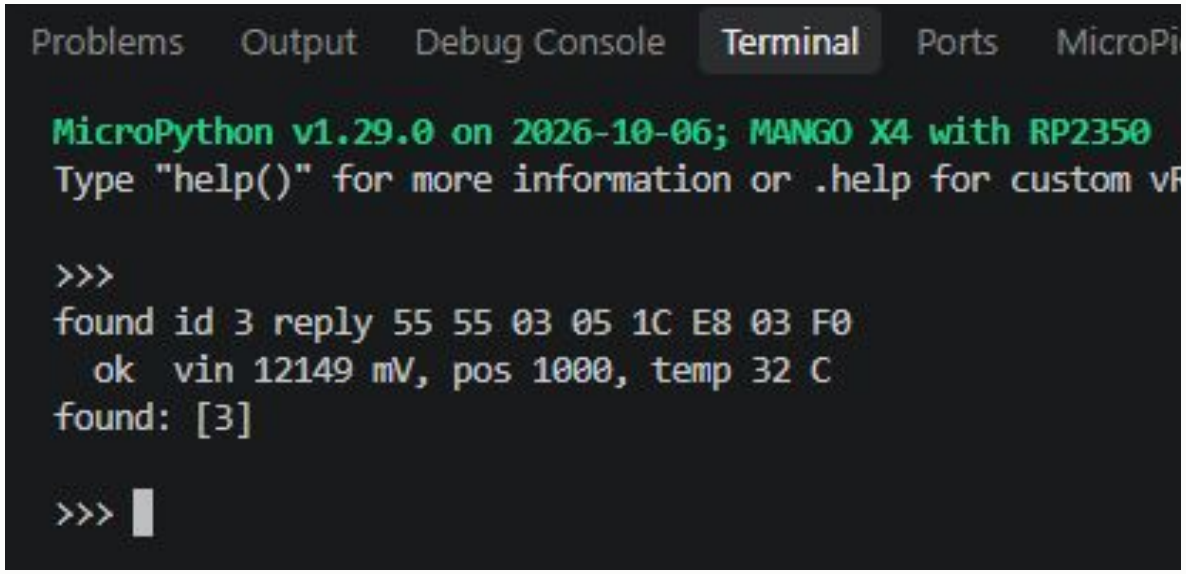


1

Power off, keep one

Same ID: keep only one connected

Change Servo ID

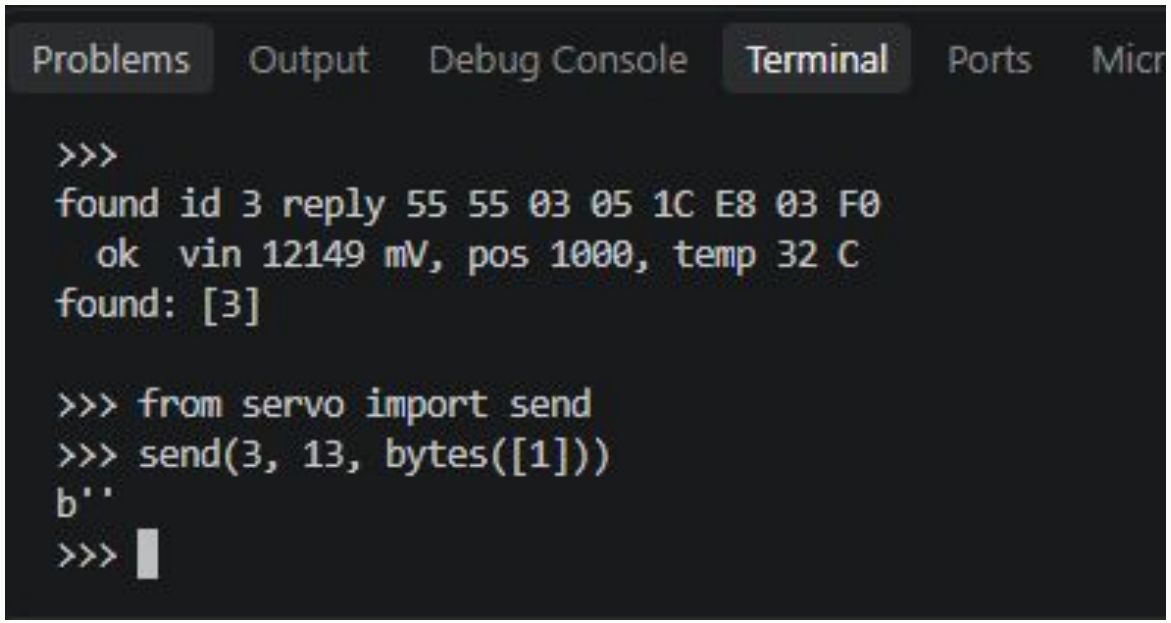


2

Power on, read it

Run scan.py

Change Servo ID



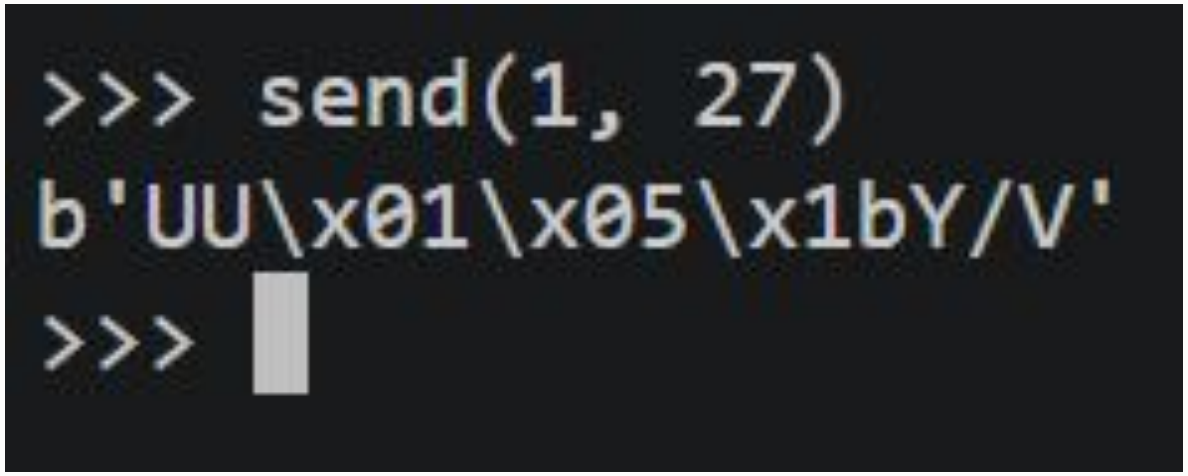
3

Send: change ID

```
from servo import send
send(3, 13, bytes([1]))
```

Change ID 3 to 1

Change Servo ID



4

Read new ID

```
send(1, 27)
# or run scan.py
```

A reply means it worked

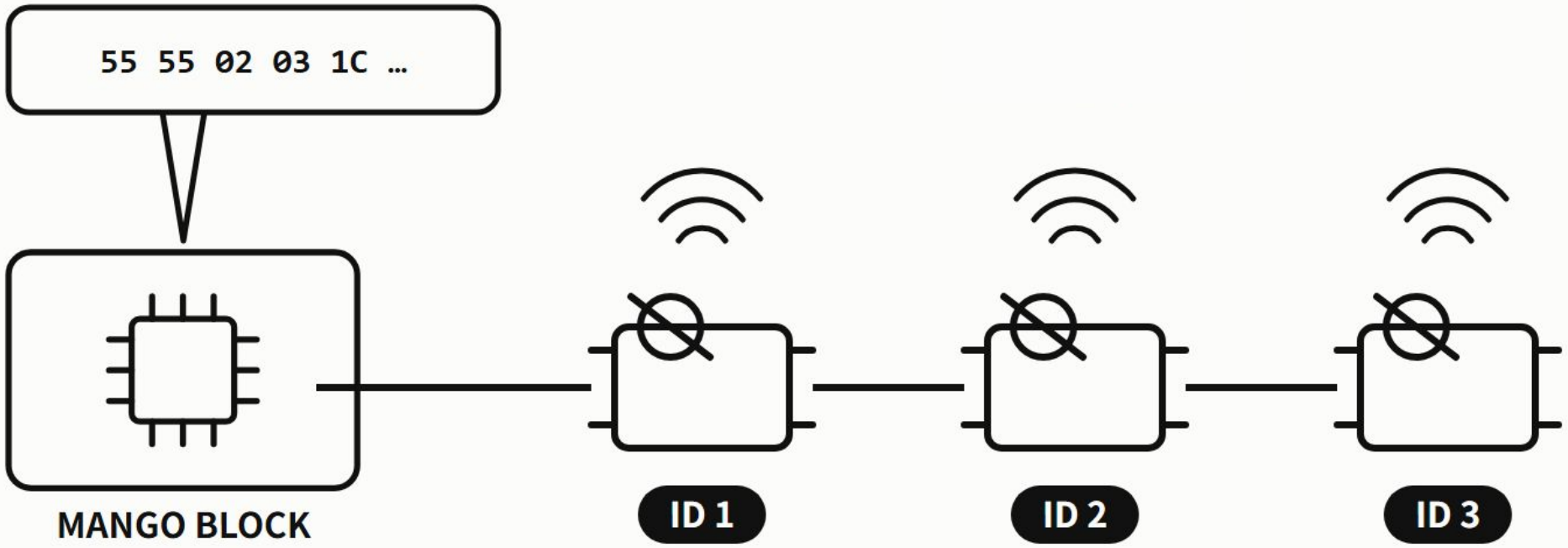
Goals

Set the three servos to IDs 1, 2, 3

Connect

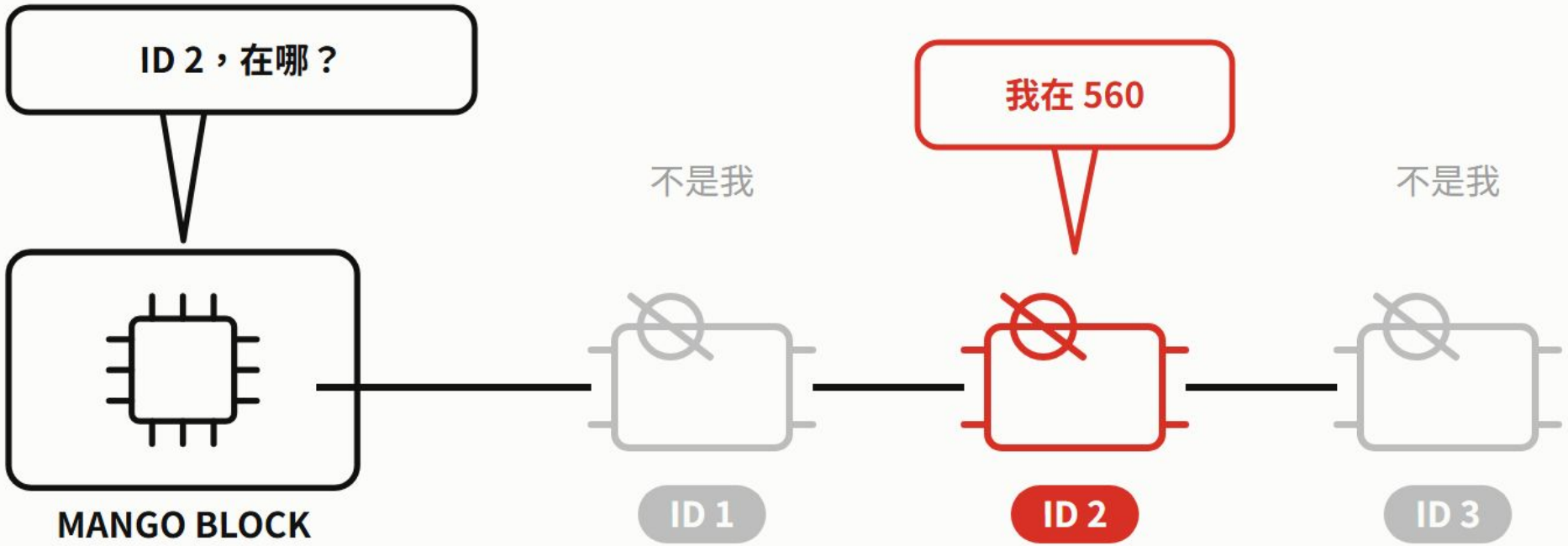

Hiwonder BUS SERVO
45 HTD-45H KG·cm
Hiwonder
45 KG·cm
Hiwonder BUS SERVO
HTD-45
45 KG·cm

Multi-servo Communication



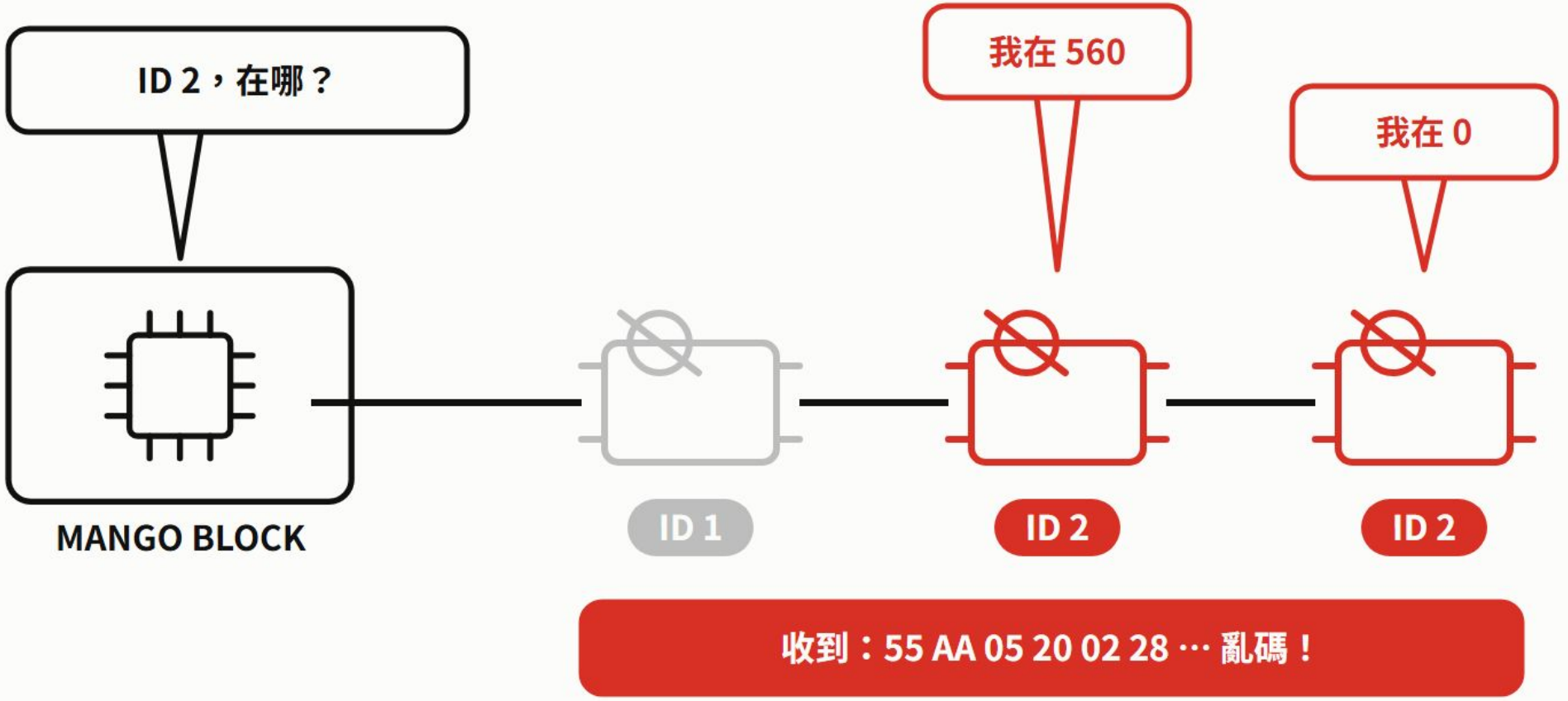
Three on one line: every servo receives every packet

Multi-servo Communication



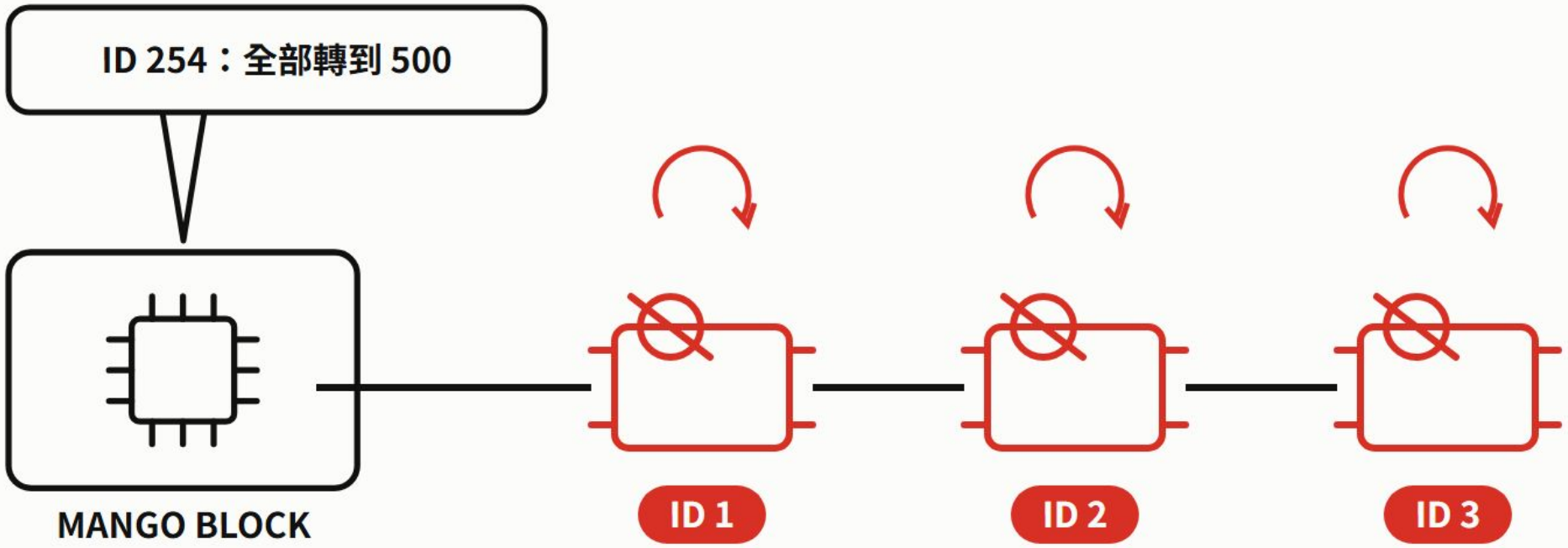
The ID in the packet decides who answers; the others ignore it

Multi-servo Communication



Two with the same ID → both answer → garbage

Multi-servo Communication



ID 254 is broadcast: all of them act, but commands only

Multi-servo Communication



Move together: send three moves in a row, all start almost at once

Move Several Together

Step by step (first from servo import *)

```
move(2, 600, 1000)
```

Turn only ID 2

```
for i in (1, 2, 3): move(i, 600, 2000)
```

All three to 600

```
for i in (1, 2, 3): move(i, 500, 2000)
```

All three back to center

```
[pos(i) for i in (1, 2, 3)]
```

Read all three positions

Write Your Own Servo Program

Write Your Own Program


```
Output of example.py
id 1 starts at 1000
    went to 951
    back at 999
id 2 starts at 0
    went to 49
    back at 1
id 3 starts at 560
    went to 511
    back at 559
```

1

Open example.py

Run it

This is an example program

The number of beeps tells which ID is moving

Each turns a little toward center, then back

Write Your Own Program

play.py

```
from servo import *


# start your own code in
here
beep()
```

2

Your turn: play.py

Use move / pos / vin / beep
to write your own program

Write Your Own Program

In play.py, finish.....

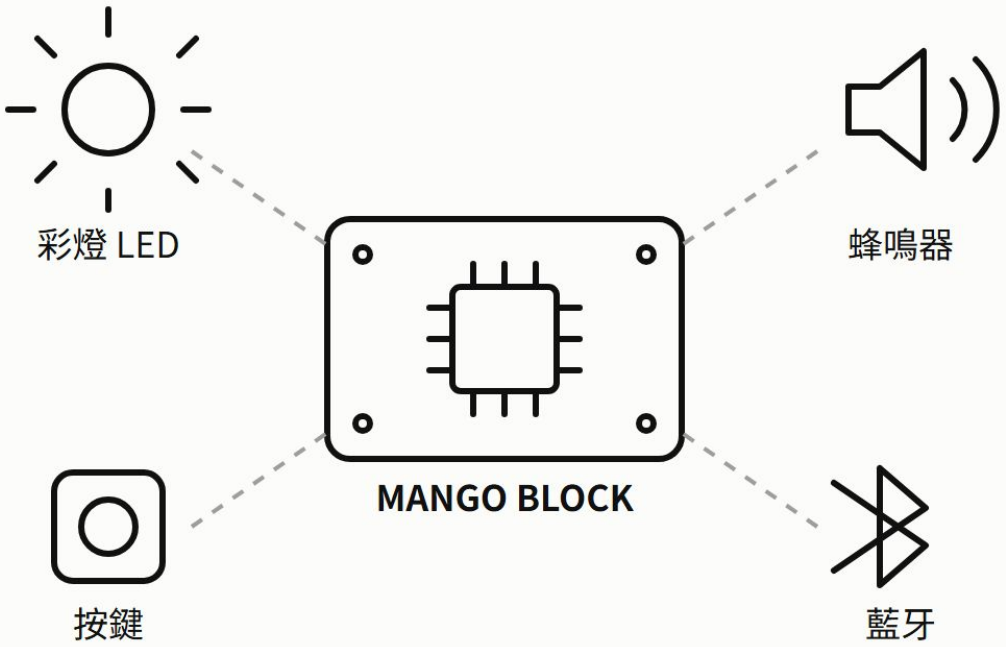
1. **Swing one servo back and forth 3 times**
2. **Turn all three at once**
3. **Control a servo with the buttons**

~ Free play ~

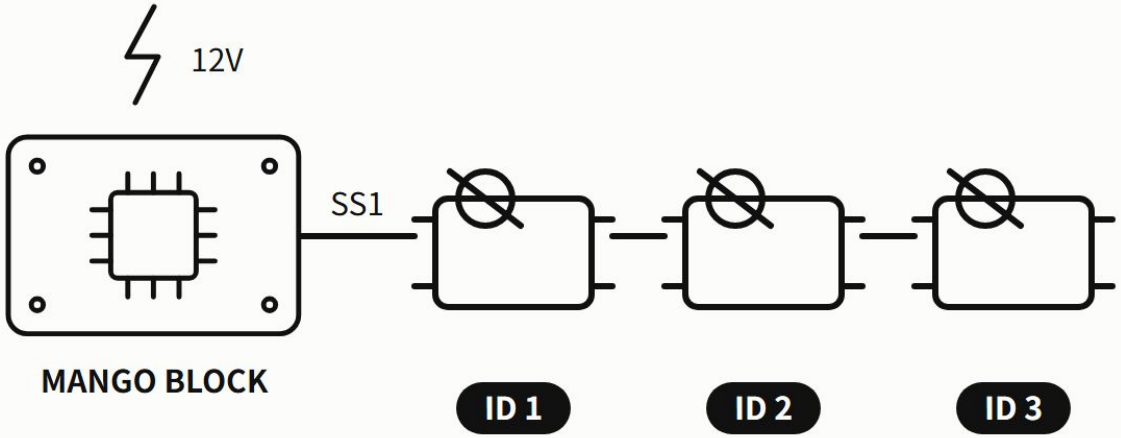
- **Power off before rewiring**
- **Secure the servo first**
- **Small angles, slow speed**

Summary

Today we learned.....



How to use MANGO BLOCK and its built-in features



Controlling servos with MANGO BLOCK

Q & A

Ask us anything, morning or afternoon